

## EXPERIMENT – 1

**OBJECTIVE:** To download and install Oracle VirtualBox on Windows and verify successful installation.

### THEORY

**Virtualization** is the process of creating a software-based (virtual) version of a computer system, including hardware, OS, storage, and network resources — all running on a single physical machine.

**Oracle VirtualBox** is a free, open-source, cross-platform **Type 2 Hypervisor** developed by Oracle Corporation. It runs on top of an existing host OS (like Windows) and allows users to create and manage multiple **Virtual Machines (VMs)**.

### Key Concepts

**Hypervisor** — Software that creates and manages VMs. Two types:

- **Type 1 (Bare-metal)** — Runs directly on hardware (e.g., VMware ESXi)
- **Type 2 (Hosted)** — Runs on top of a host OS (e.g., VirtualBox)

**Host Machine** — The physical computer on which VirtualBox is installed.

**Guest Machine** — The virtual machine running inside VirtualBox.

**Guest Additions** — A set of drivers installed inside the guest OS to improve performance (shared clipboard, seamless mode, better display resolution).

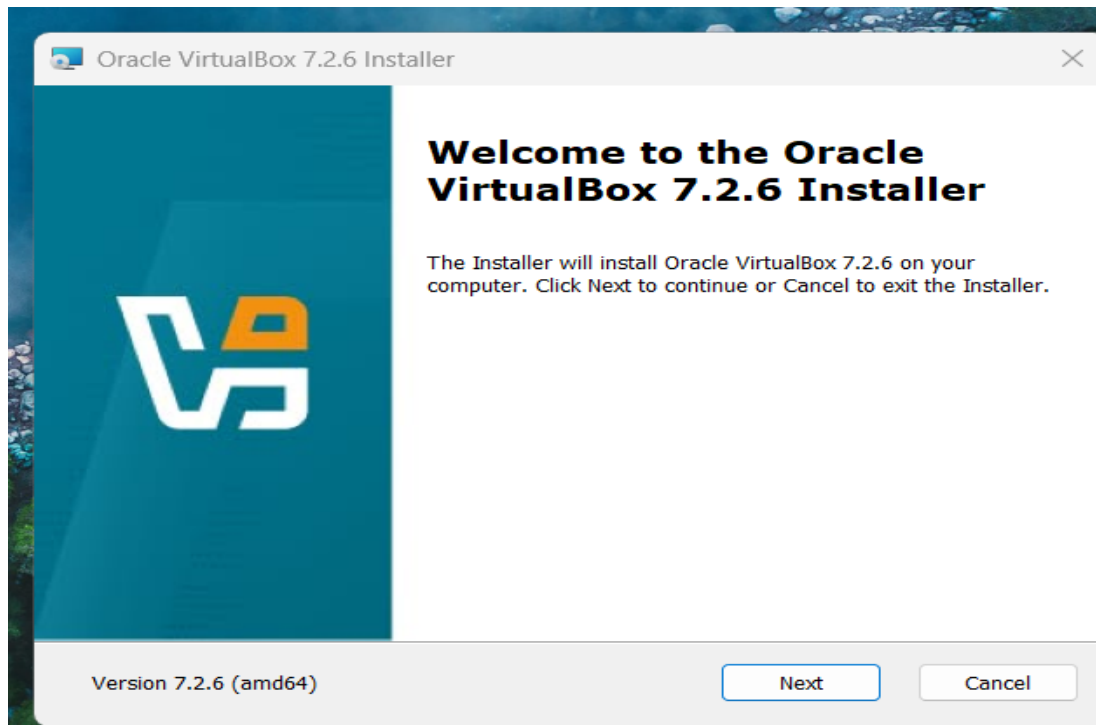
### Features VirtualBox

- Free and open-source (GNU GPL)
- Supports multiple guest OS: Windows, Linux, macOS, Solaris
- Snapshot feature to save/restore VM states
- Isolated environment — safe for testing without affecting the host
- Supports 32-bit and 64-bit guest systems

### PROCEDURE

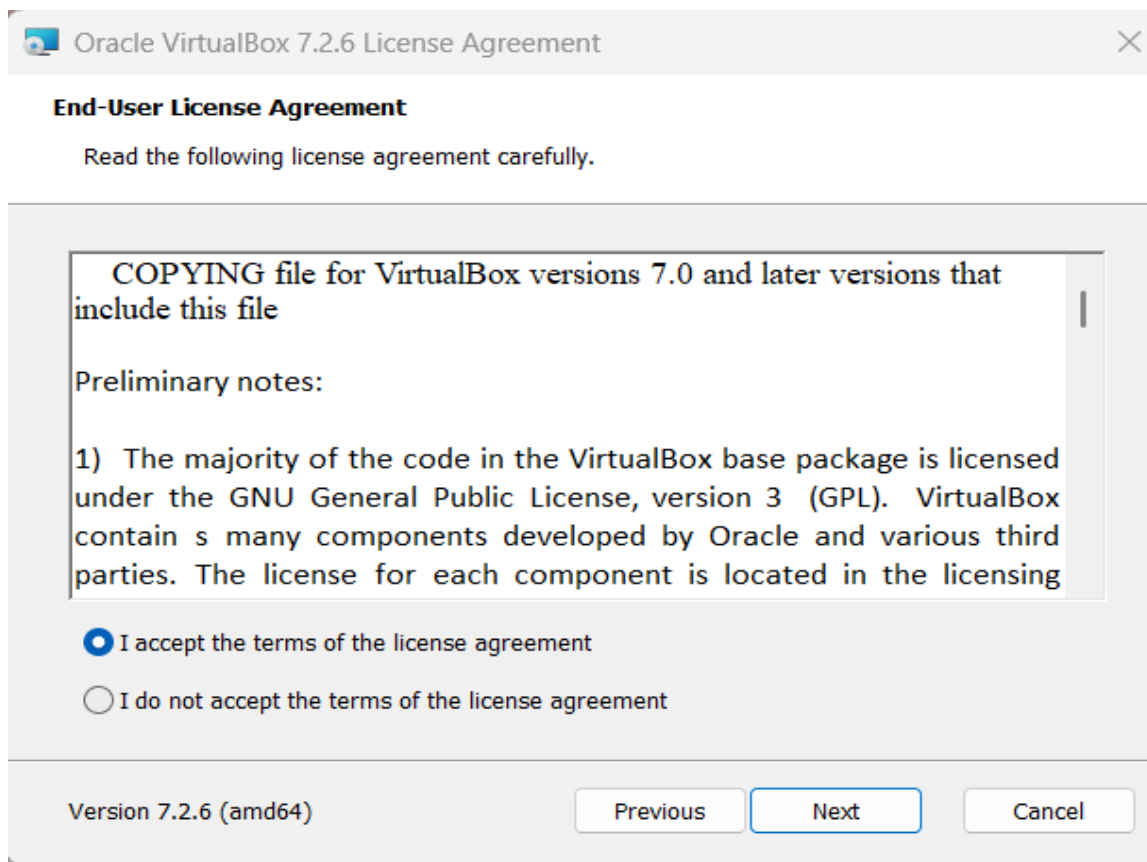
#### Step 1: Launch Installer

Open the VirtualBox setup file. The Welcome screen appears showing “Welcome to the Oracle VirtualBox Installer”. Click Next to continue.



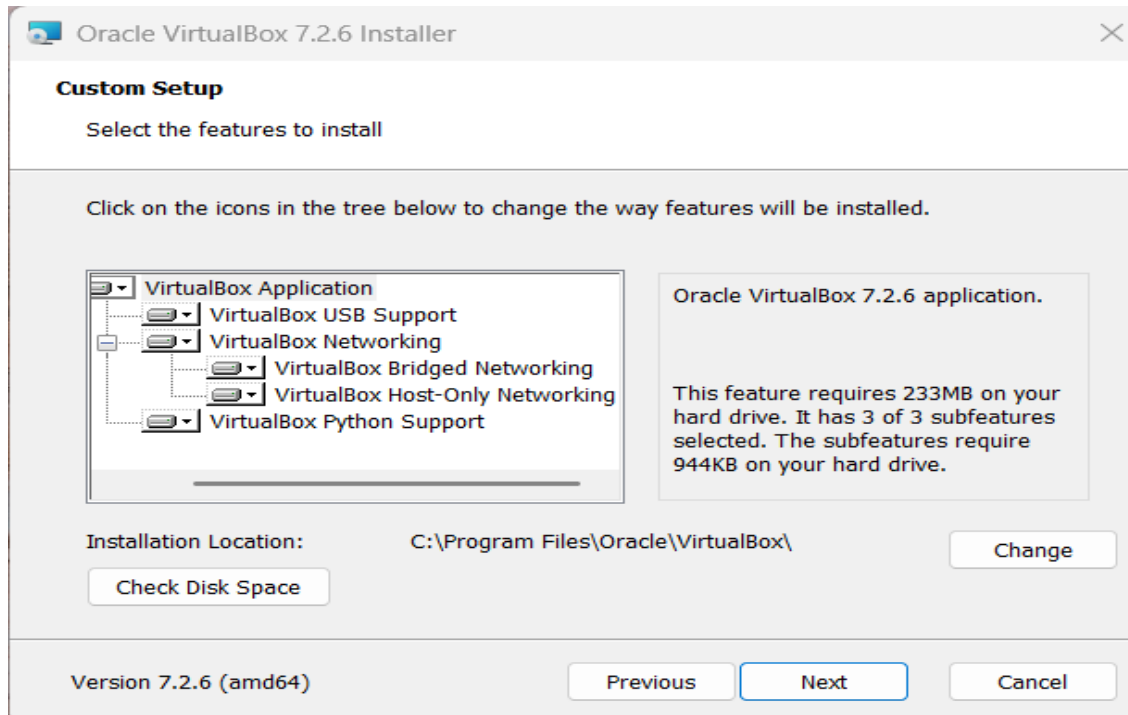
## Step 2: License Agreement

Read the End-User License Agreement, select “I accept the terms of the license agreement”, and click Next.



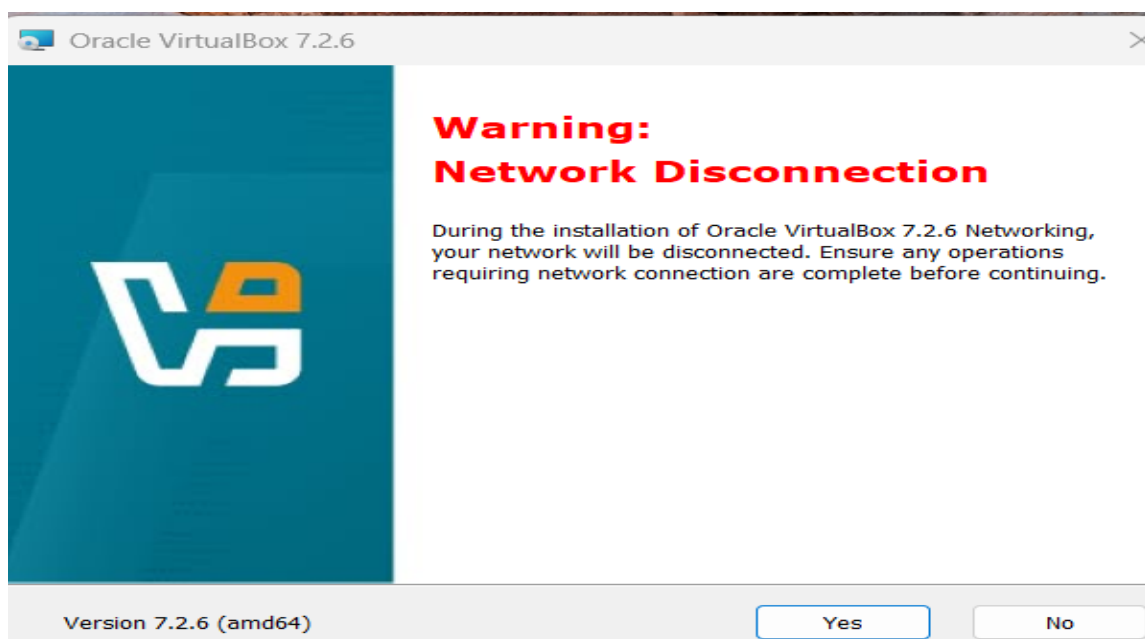
### Step 3: Custom Setup – Feature Selection

The setup displays components such as VirtualBox Application, USB Support, Networking Features, and Python Support. Keep default selections unless specific changes are needed. Choose installation location if required, then click Next.

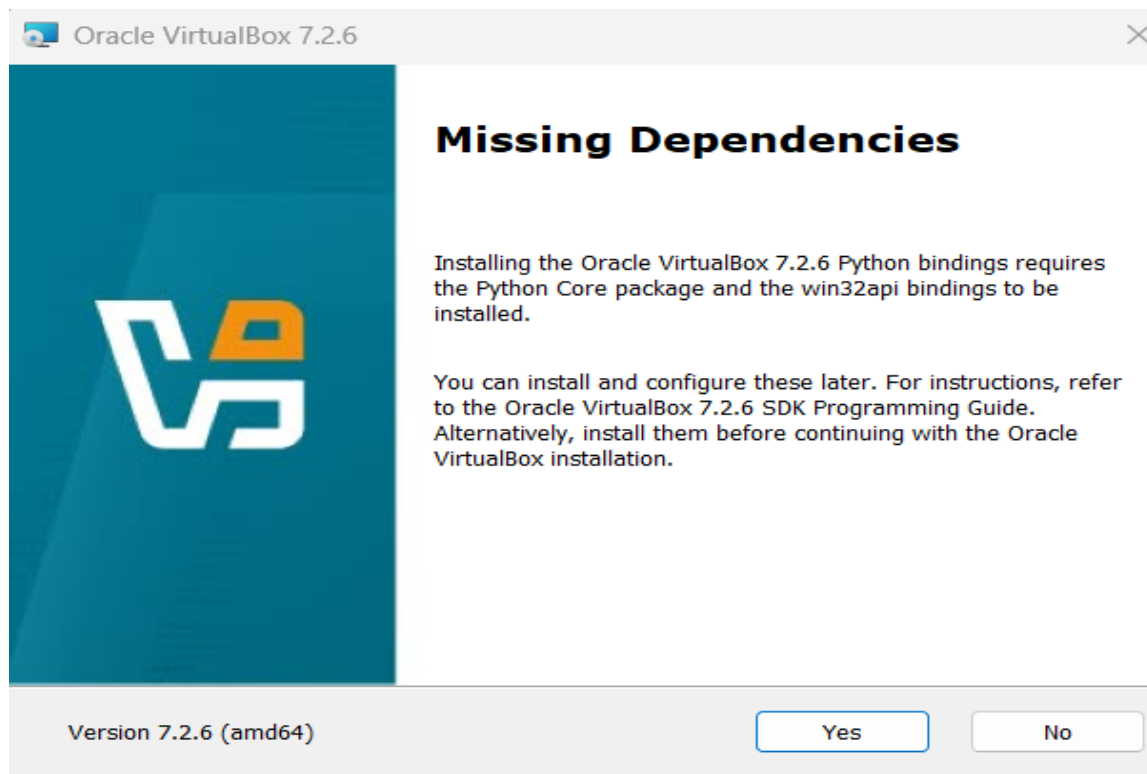


### Step 4: Network Interface Warning

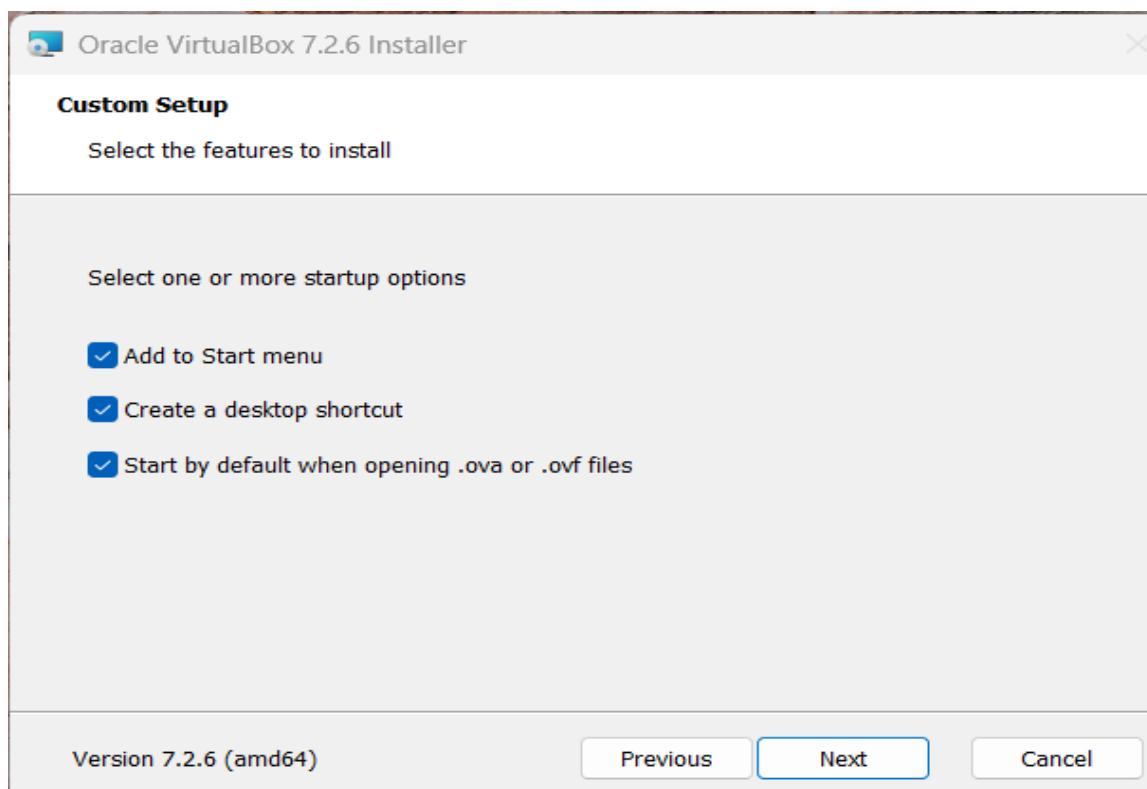
A warning message appears stating that network connection may be temporarily disconnected during installation. Click Yes to proceed.



## Step 5: Missing dependencies

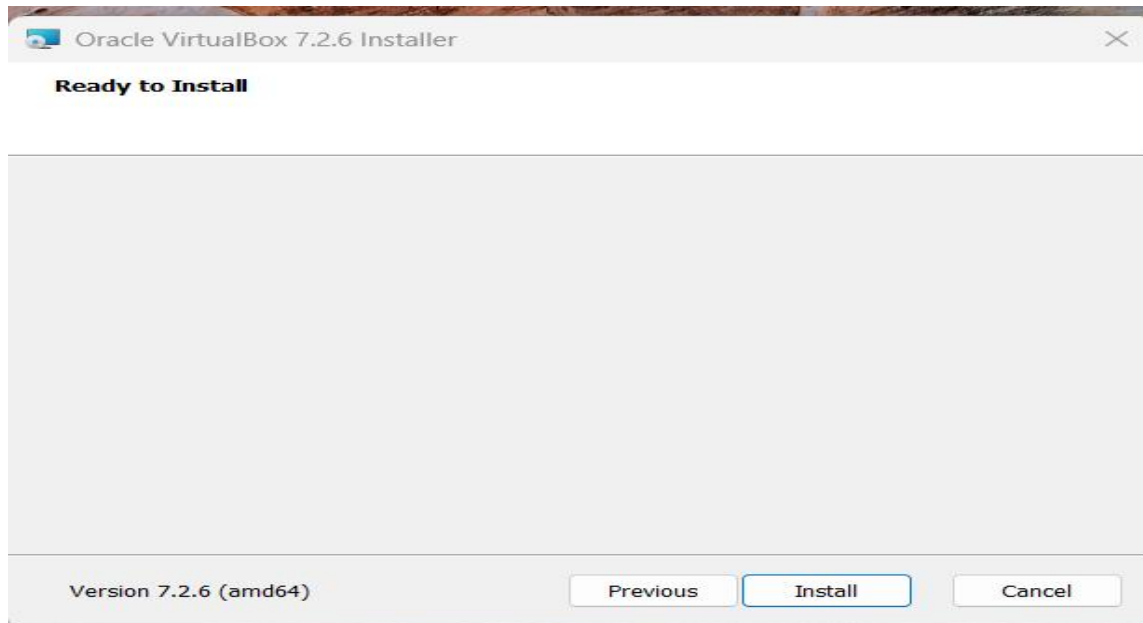


## Step 6: Custom Setup



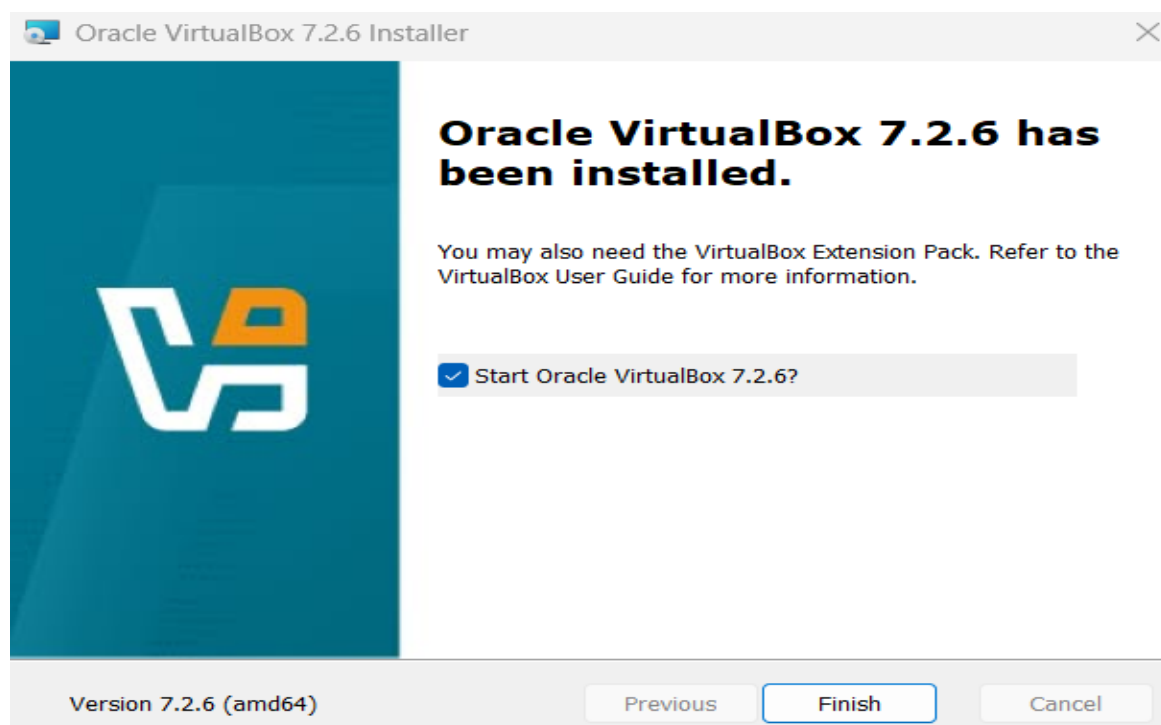
### Step 7: Ready to Install

The installer shows the Ready to Install screen. Click Install to begin installation  
Installation Completion



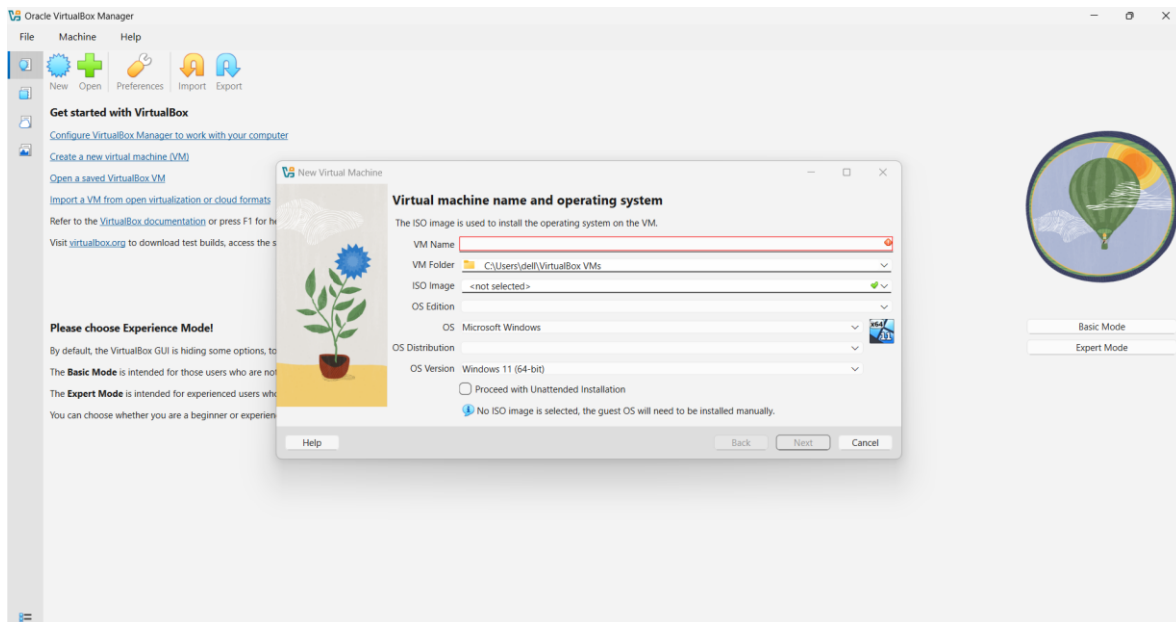
### Step 8: Successful installation

Wait for installation to finish. A confirmation message appears indicating that VirtualBox has been installed successfully. Keep “Start Oracle VirtualBox” checked and click Finish.



## Step 9: Launching VirtualBox

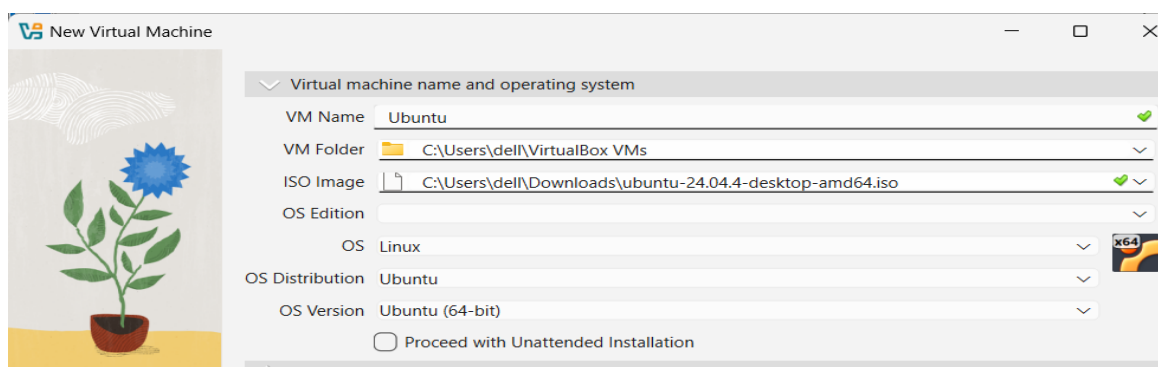
The VirtualBox Manager window opens. Choose Basic Mode or Expert Mode. The software is now ready to create and manage virtual machines.



## Step 10: Create a New Virtual Machine

Click the "New" button in VirtualBox Manager. The "Virtual machine name and operating system" screen appears. Fill in the following details:

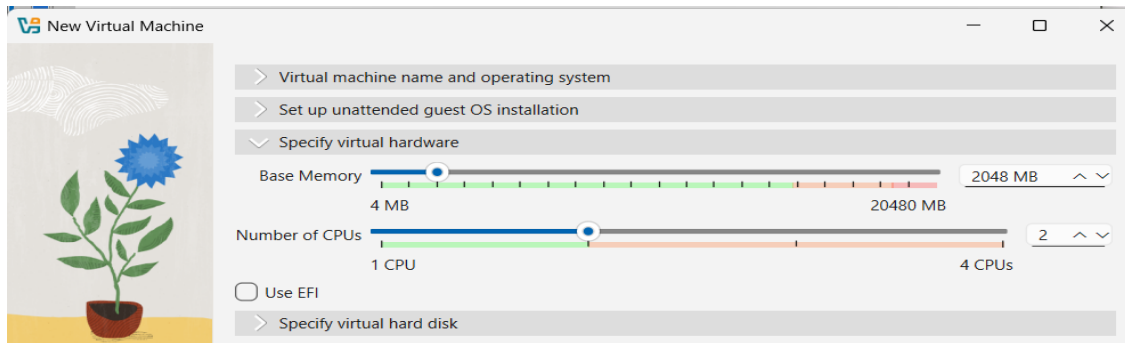
- VM Name: Ubuntu
- VM Folder: C:\Users\dell\VirtualBox VMs
- ISO Image: Select the downloaded Ubuntu 24.04 LTS ISO file
- OS: Linux
- OS Version: Ubuntu (64-bit)
- Proceed with Unattended Installation: ☐ Unchecked



## Step 11: Specify Virtual Hardware

Expand the "Specify virtual hardware" section and configure:

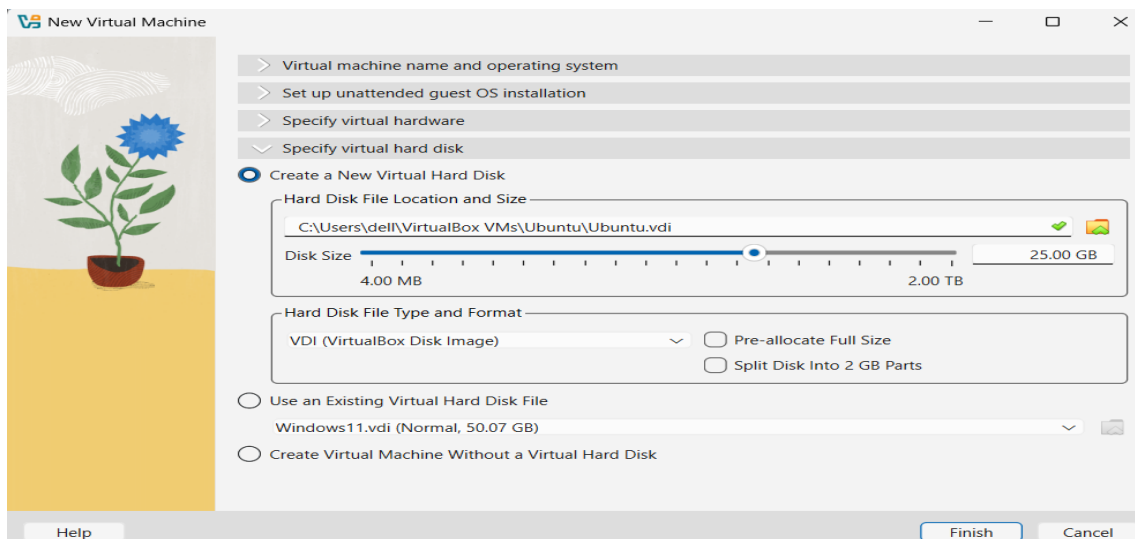
- Base Memory: 2048 MB
- Number of CPUs: 2
- Use EFI: ☐ Unchecked (*Ubuntu does not require EFI*)



## Step 12 : Specify Virtual hard disk

Expand the "Specify virtual hard disk" section and configure:

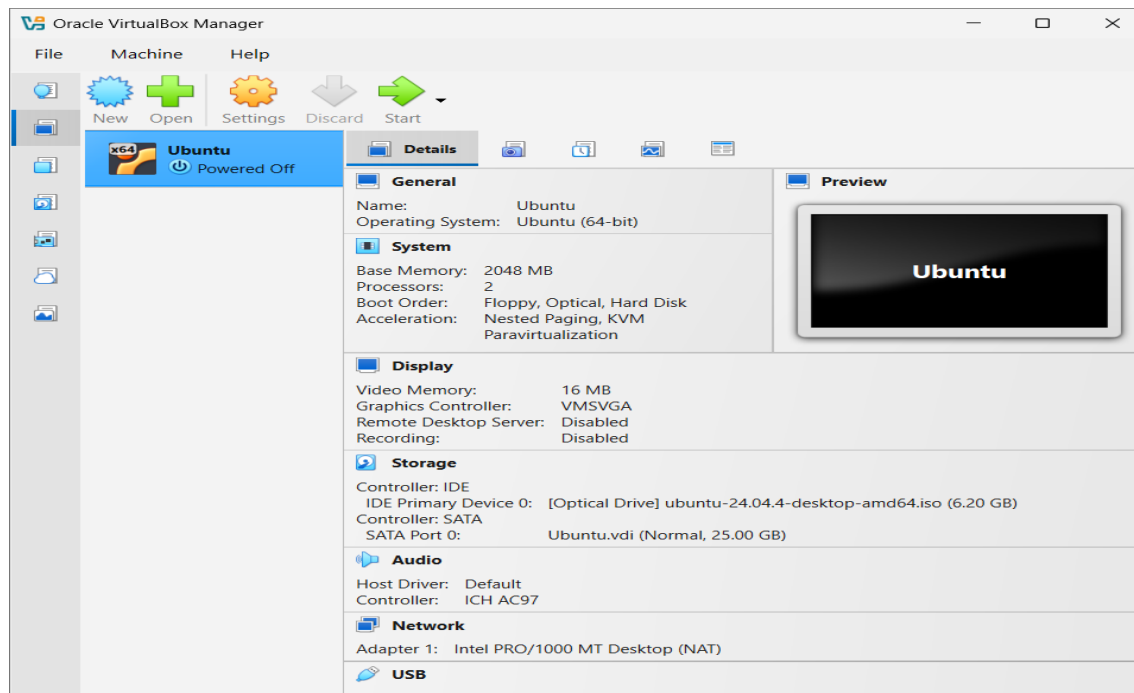
- Create a New Virtual Hard Disk: Selected
- Disk Size: 25 GB
- Hard Disk File Type: VDI (VirtualBox Disk Image)



## Step 13: Virtual Machine Created Successfully

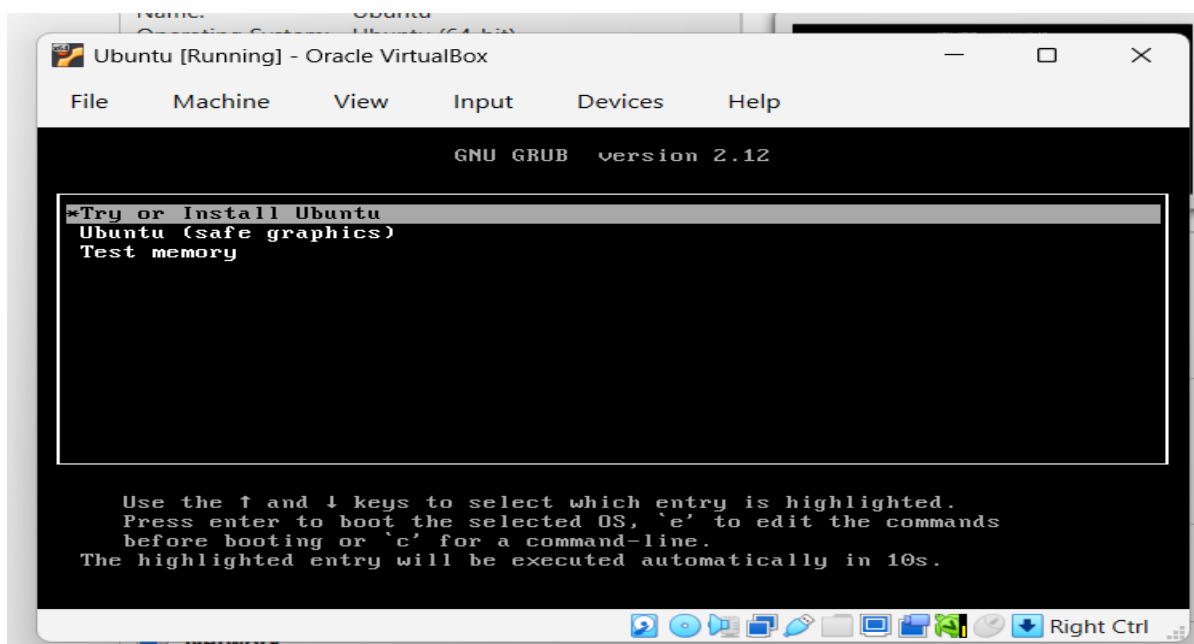
After clicking Finish, the Oracle VM VirtualBox Manager displays the newly created Ubuntu VM with the following confirmed configuration:

- **Name:** Ubuntu
- **Operating System:** Ubuntu (64-bit)
- **Base Memory:** 2048 MB
- **Processors:** 2
- **Hard Disk:** Ubuntu.vdi (25 GB)
- **ISO Attached:** ubuntu-24.04-desktop-amd64.iso



### Step 14: Start the Virtual Machine

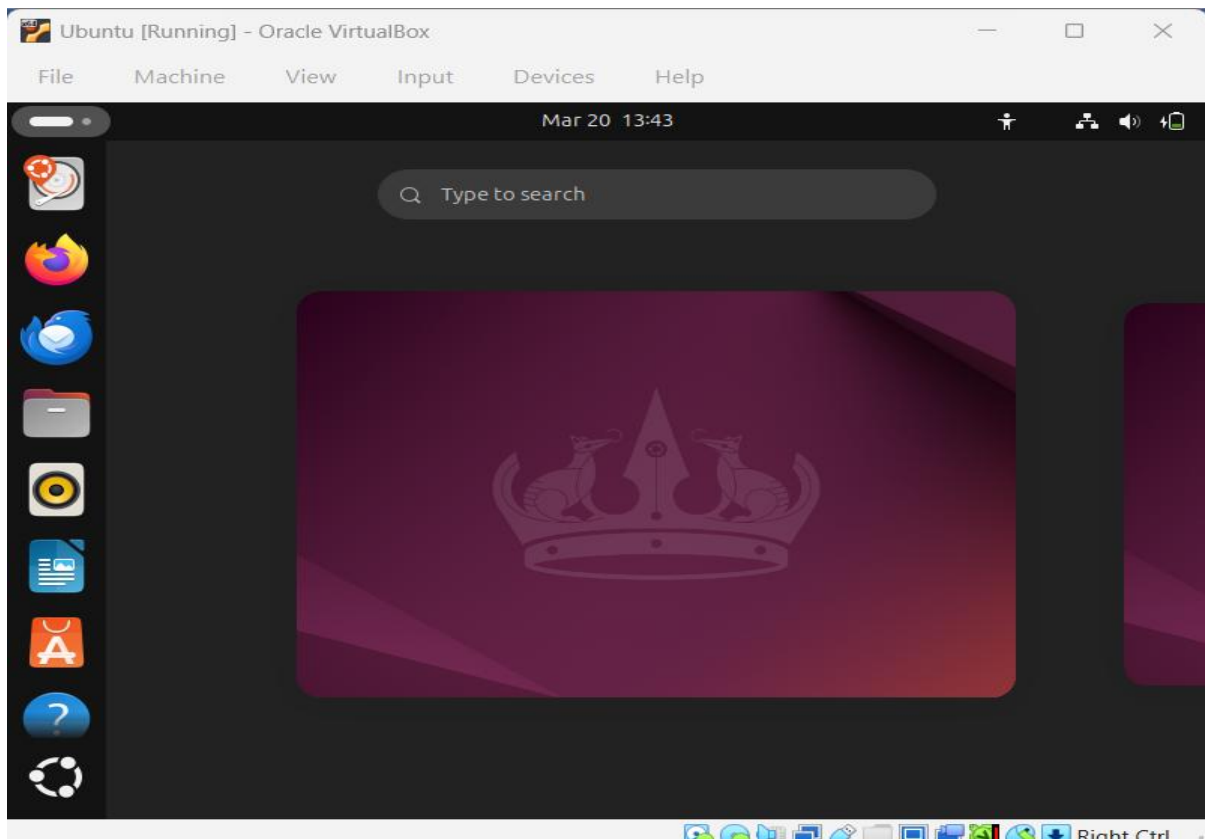
Click the green "Start" button in the VirtualBox Manager toolbar. The VM status changes to "Running" in the left panel.





### Step 15: Ubuntu Setup Loading

The VM window opens and Ubuntu boots from the ISO. The Ubuntu installation/welcome screen appears inside the VirtualBox window, confirming that a completely different OS (Linux) is running on the same physical machine as the host Windows 11.



### RESULT

Oracle VirtualBox was successfully installed on the Windows 11 host machine. An Ubuntu Linux Virtual Machine was created and successfully booted inside VirtualBox. The Ubuntu desktop/installation screen appeared correctly inside the VM window, confirming that a completely different Operating System (Ubuntu Linux) can run simultaneously on the same physical machine alongside Windows 11 using virtualization technology.

## EXPERIMENT – 2

**OBJECTIVE:** Install a C Compiler in the virtual machine created using virtual box and execute any simple program

### REQUIREMENTS

- Ubuntu VM with GCC installed (from Experiment 3)
- Terminal access
- nano text editor (pre-installed on Ubuntu)

### THEORY

**C Programming Language** is a general-purpose, procedural programming language developed by Dennis Ritchie at Bell Labs in 1972. It is one of the most widely used programming languages and forms the foundation of many modern languages like C++, Java, and Python.

**GCC (GNU Compiler Collection)** is a free, open-source compiler system developed by the GNU Project. It supports multiple programming languages including C, C++, Fortran, and others. GCC is the standard compiler for Linux/Ubuntu systems.

### Key Concepts:

**Compiler** — A compiler is a program that translates source code written in a high-level language (like C) into machine code (binary) that the computer's processor can execute directly.

**Compilation Process** — The C compilation process involves 4 stages:

- **Preprocessing** — Handles directives like #include and #define
- **Compilation** — Converts C code to assembly language
- **Assembly** — Converts assembly to machine code (object file)
- **Linking** — Links object files to create the final executable

**Terminal** — A command-line interface in Ubuntu/Linux used to execute commands, install software, and run programs.

**apt (Advanced Package Tool)** — The package manager used in Ubuntu/Debian-based Linux systems to install, update, and remove software packages.

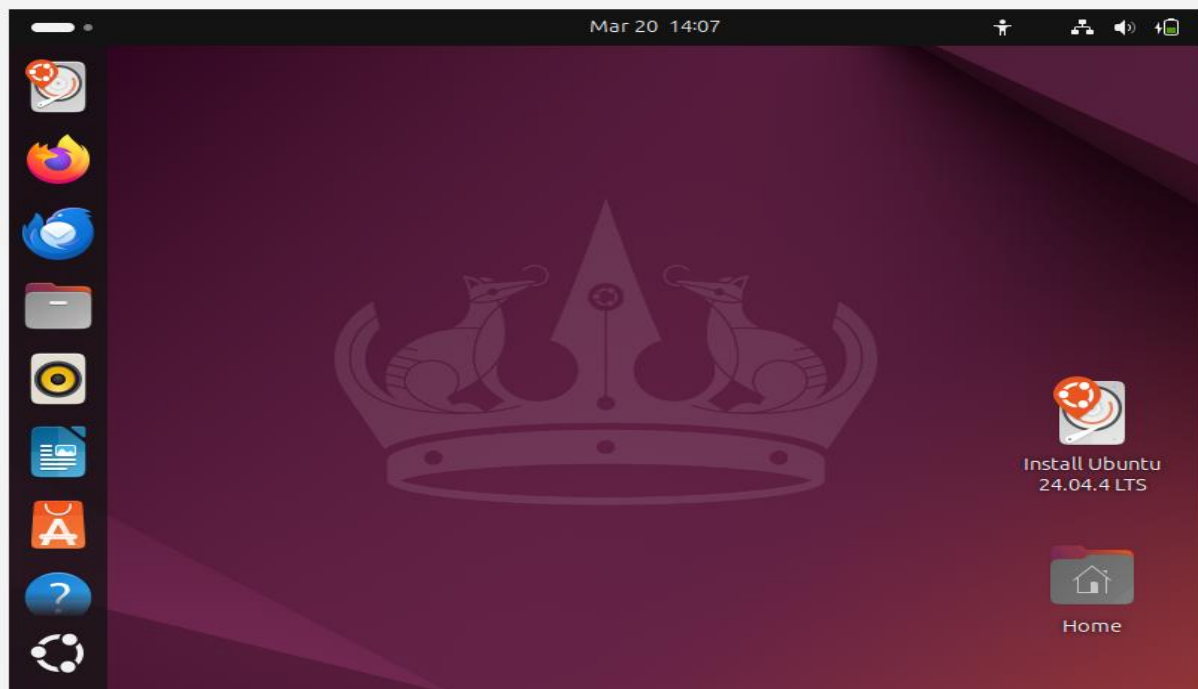
### Why GCC on Ubuntu VM?

- GCC is the most widely used C compiler on Linux
- Ubuntu provides GCC through its official package repository
- Installing inside a VM keeps the host system clean and isolated

## PROCEDURE

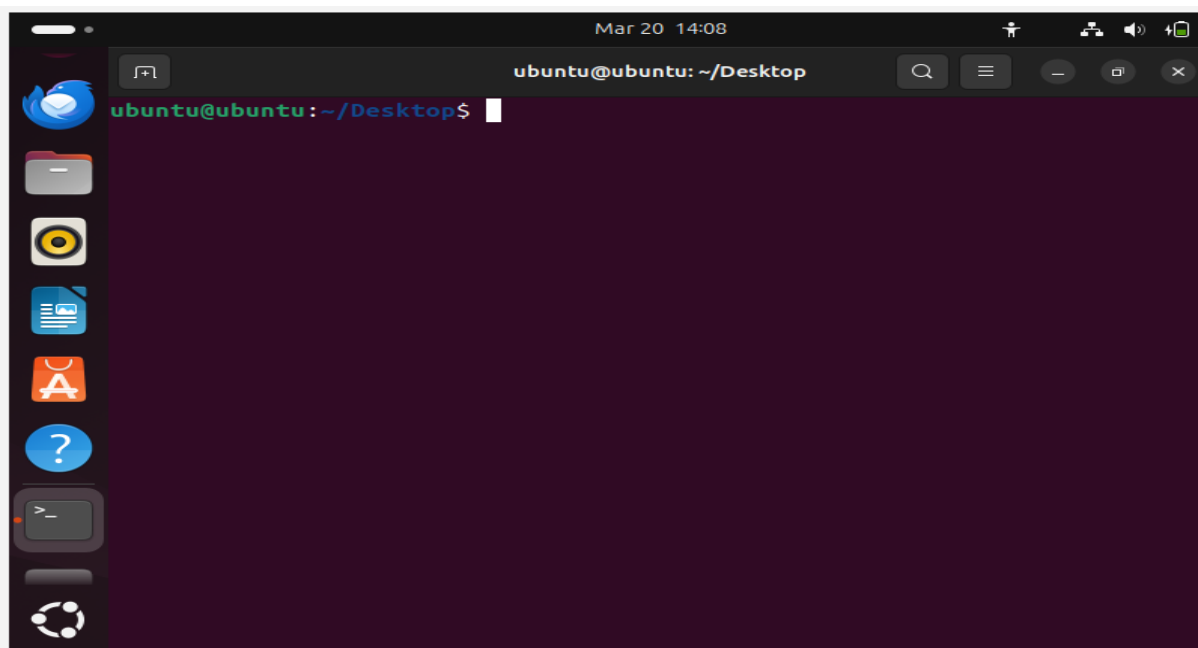
### Step 1: Open Ubuntu VM

Launch Oracle VirtualBox Manager and start the Ubuntu Virtual Machine. Wait for the Ubuntu desktop to load completely inside the VM window.



### Step 2: Open Terminal

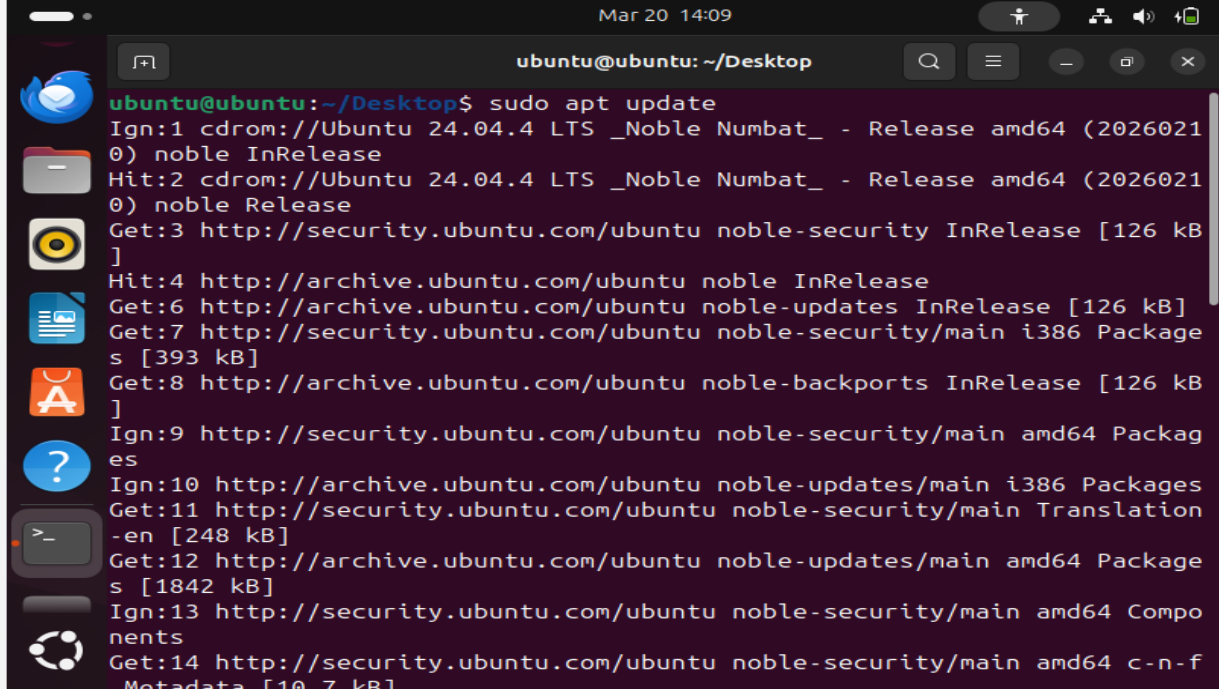
Right-click on the Ubuntu desktop and select "**Open Terminal**". The Ubuntu Terminal window opens showing the command prompt.



### Step 3: Update Package List

Before installing GCC, update the package list to ensure the latest version is available. Type the following command and press Enter:

```
sudo apt update
```



```

ubuntu@ubuntu: ~/Desktop
ubuntu@ubuntu:~/Desktop$ sudo apt update
Ign:1 cdrom://Ubuntu 24.04.4 LTS _Noble Numbat_ - Release amd64 (20260210) noble InRelease
Hit:2 cdrom://Ubuntu 24.04.4 LTS _Noble Numbat_ - Release amd64 (20260210) noble Release
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease
Get:6 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main i386 Packages [393 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Ign:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages
Ign:10 http://archive.ubuntu.com/ubuntu noble-updates/main i386 Packages
Get:11 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [248 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1842 kB]
Ign:13 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components
Get:14 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]

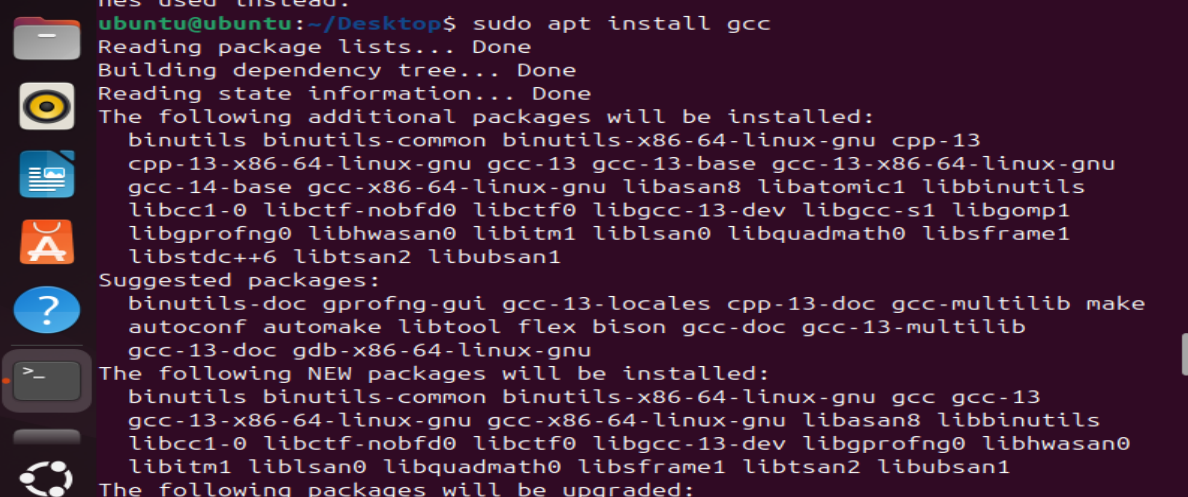
```

### Step 4: Install GCC C Compiler

Type the following command and press Enter to install GCC:

```
sudo apt install gcc
```

When prompted **"Do you want to continue? [Y/n]"** type **Y** and press Enter. Wait for the installation to complete.



```

ubuntu@ubuntu:~/Desktop$ sudo apt install gcc
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cpp-13
  cpp-13-x86-64-linux-gnu gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu
  gcc-14-base gcc-x86-64-linux-gnu libasan8 libatomic1 libbinutils
  libcc1-0 libctf-nobfd0 libctf0 libgcc-13-dev libgcc-s1 libgomp1
  libgprofng0 libhwasan0 libitm1 liblsan0 libquadmath0 libsframe1
  libstdc++6 libtsan2 libubsan1
Suggested packages:
  binutils-doc gprofng-gui gcc-13-locales cpp-13-doc gcc-multilib make
  autoconf automake libtool flex bison gcc-doc gcc-13-multilib
  gcc-13-doc gdb-x86-64-linux-gnu
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu gcc gcc-13
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libasan8 libbinutils
  libcc1-0 libctf-nobfd0 libctf0 libgcc-13-dev libgprofng0 libhwasan0
  libitm1 liblsan0 libquadmath0 libsframe1 libtsan2 libubsan1
The following packages will be upgraded:

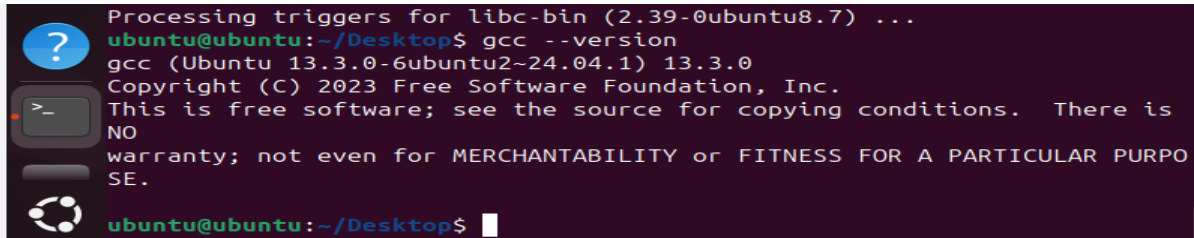
```

### Step 5: Verify GCC Installation

After installation, verify that GCC is installed correctly by checking its version:

```
gcc --version
```

The terminal displays the installed GCC version, confirming successful installation.

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop\$'. The command 'gcc --version' has been executed, and the output is displayed in green and white text. The output includes the GCC version (13.3.0), the Ubuntu version (13.3.0-6ubuntu2~24.04.1), and a copyright notice for the Free Software Foundation, Inc. The prompt returns to 'ubuntu@ubuntu:~/Desktop\$' after the command.

```
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
ubuntu@ubuntu:~/Desktop$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is
NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPO
SE.
ubuntu@ubuntu:~/Desktop$
```

### Step 6: Create a C Program

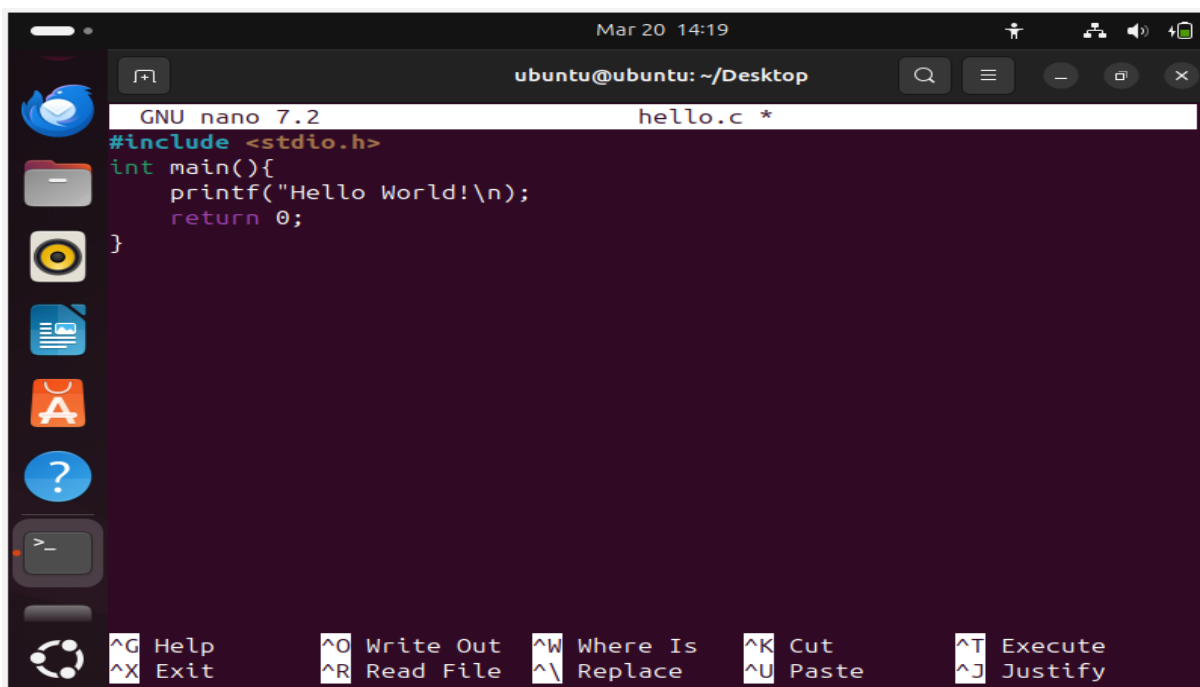
Create a new C source file using the nano text editor:

```
nano hello.c
```

The nano editor opens. Type the following simple C program:

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Press **\*\*Ctrl + X\*\*** → Press **\*\*Y\*\*** → Press **\*\*Enter\*\*** to save and exit.

A screenshot of a terminal window with a dark purple background. The window title is 'ubuntu@ubuntu: ~/Desktop'. The terminal shows the 'GNU nano 7.2' editor interface. The file 'hello.c' is open, and the following C code is visible: '#include <stdio.h>', 'int main(){', 'printf("Hello World!\n");', 'return 0;', and '}'. The bottom of the screen shows a row of keyboard shortcuts: '^G Help', '^X Exit', '^O Write Out', '^R Read File', '^W Where Is', '^\_ Replace', '^K Cut', '^U Paste', '^T Execute', and '^J Justify'. The system clock in the top right corner shows 'Mar 20 14:19'.

```
Mar 20 14:19
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2 hello.c *
#include <stdio.h>
int main(){
    printf("Hello World!\n");
    return 0;
}
^G Help    ^O Write Out  ^W Where Is  ^K Cut      ^T Execute
^X Exit    ^R Read File  ^_ Replace   ^U Paste    ^J Justify
```

### Step 7: Compile the C Program

Compile the C program using GCC:

```
gcc hello.c -o hello
```

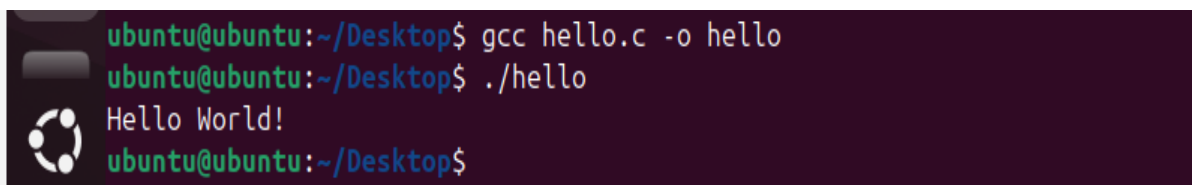
If no errors appear, the program compiled successfully and an executable file named **\*\*"hello"\*\*** is created.

### Step 8: Execute the C Program

Run the compiled program:

```
./hello
```

The output appears on the terminal:

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop\$'. The first command is 'gcc hello.c -o hello'. The second command is './hello'. The output is 'Hello World!'. The prompt returns to 'ubuntu@ubuntu:~/Desktop\$'.

## RESULT

GCC C Compiler was successfully installed inside the Ubuntu Virtual Machine running in Oracle VirtualBox. A simple C program was written, compiled using the gcc command, and executed successfully. The program displayed the output "Hello, World!" on the terminal, confirming that the C compiler is working correctly inside the Virtual Machine.

## CONCLUSION

This experiment successfully demonstrated the installation and use of a C Compiler (GCC) inside a Virtual Machine. Ubuntu Linux running inside Oracle VirtualBox provided a complete development environment for writing and executing C programs. The experiment highlights the practical use of Virtual Machines as isolated development environments where software can be installed and tested without affecting the host operating system. GCC is a powerful and widely used compiler that supports the complete C compilation process from source code to executable output.

## EXPERIMENT – 3

**OBJECTIVE:** To install Google App Engine and create a simple web application using Python and Flask demonstrating the Real-Time Fire NOC Monitoring System applicant dashboard.

### THEORY

**Google App Engine (GAE)** is a fully managed, serverless Platform-as-a-Service (PaaS) offered by Google Cloud. It allows developers to build and deploy web applications without managing the underlying infrastructure. App Engine automatically handles load balancing, scaling, and server maintenance.

#### Key features of Google App Engine:

- Supports multiple languages including Python, Java, Node.js, Go, and PHP
- Automatic scaling based on traffic
- Built-in security and monitoring
- Free tier available for small applications
- Seamless integration with other Google Cloud services like Firestore, Cloud SQL, and Cloud Storage

**Flask** is a lightweight micro web framework for Python. It is widely used for building web applications and REST APIs. Flask follows the WSGI (Web Server Gateway Interface) standard and uses Jinja2 as its templating engine, which allows dynamic HTML generation using Python variables directly inside HTML files.

**The Fire NOC Monitoring System** is a real-time web application designed to digitize the process of applying for and managing Fire No Objection Certificates (NOC). The applicant dashboard displays KPI cards (total applications, pending, approved, active NOCs), recent application history, fee summaries, upcoming inspections, and announcements — all rendered dynamically using Flask and Jinja2 templates.

#### Working of Google App Engine

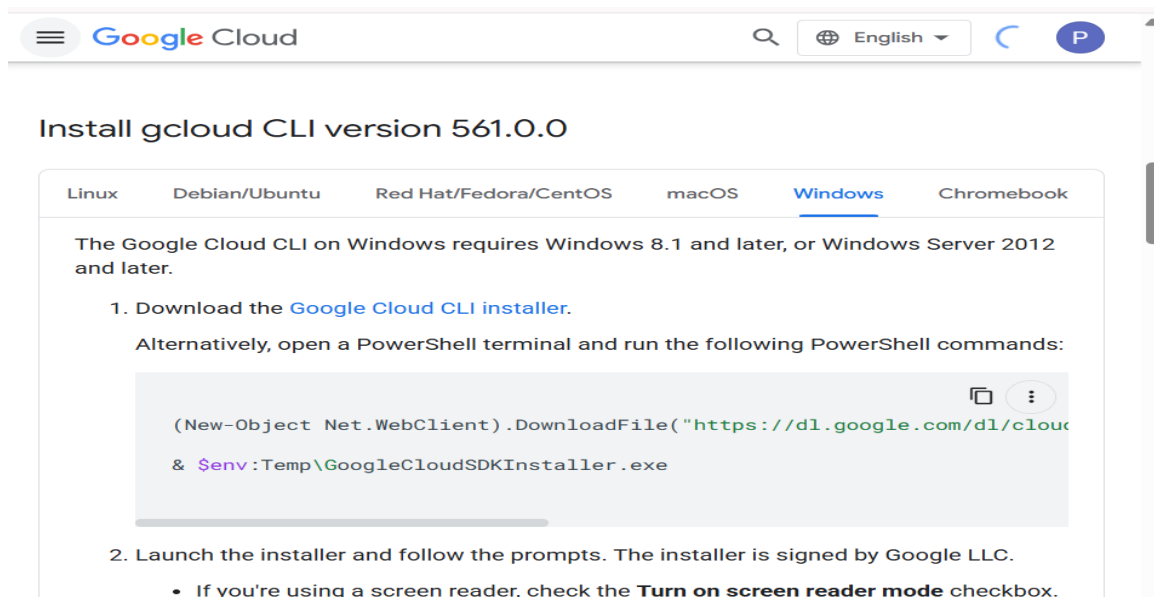
When a user visits the deployed URL, Google's infrastructure routes the request to the App Engine instance. The Flask application processes the request, queries the database if needed, renders the HTML template with dynamic data, and sends the response back to the user. App Engine manages all scaling and availability automatically.

### PROCEDURE

#### Step 1: Download and Run the Installer

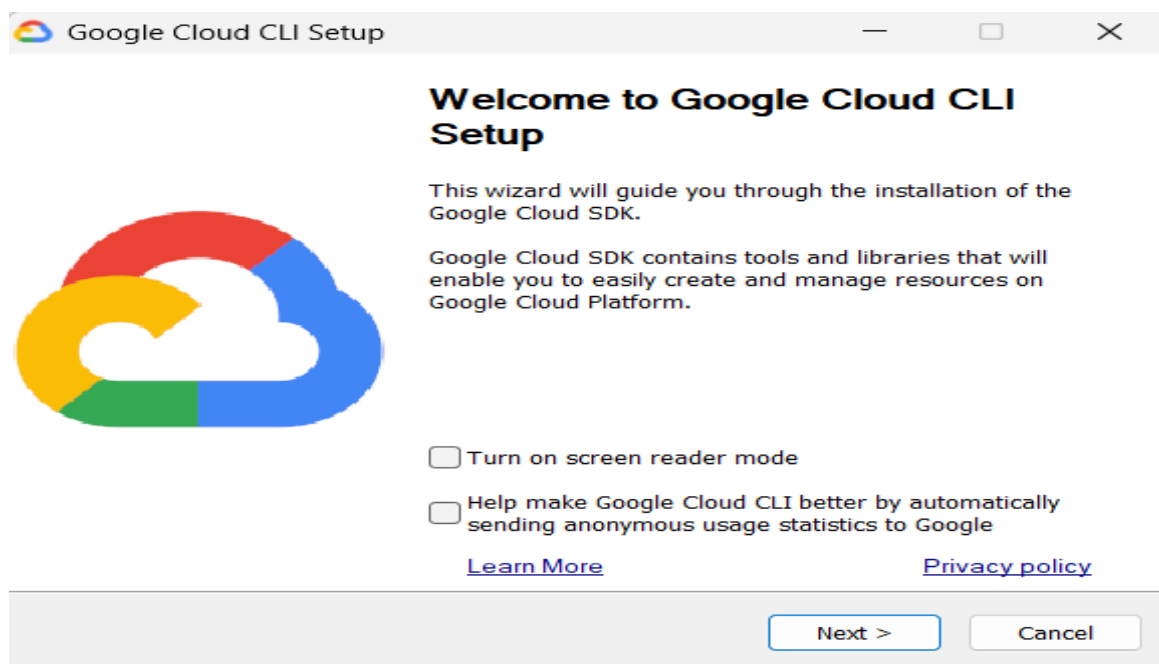
Open the browser and go to the following direct download link:

<https://dl.google.com/dl/cloudsdk/channels/rapid/GoogleCloudSDKInstaller.exe>  
This directly downloads the installer file GoogleCloudSDKInstaller.exe to the Downloads folder.



## Step 2: Installation Wizard

Go to the Downloads folder and double click GoogleCloudSDKInstaller.exe. Click Yes when Windows asks for permission. The installation wizard opens. Click Next, Next, Next, and Install. Wait for installation to complete then click Finish. After clicking Finish, a black command window opens automatically showing the Google Cloud CLI welcome message confirming successful installation.



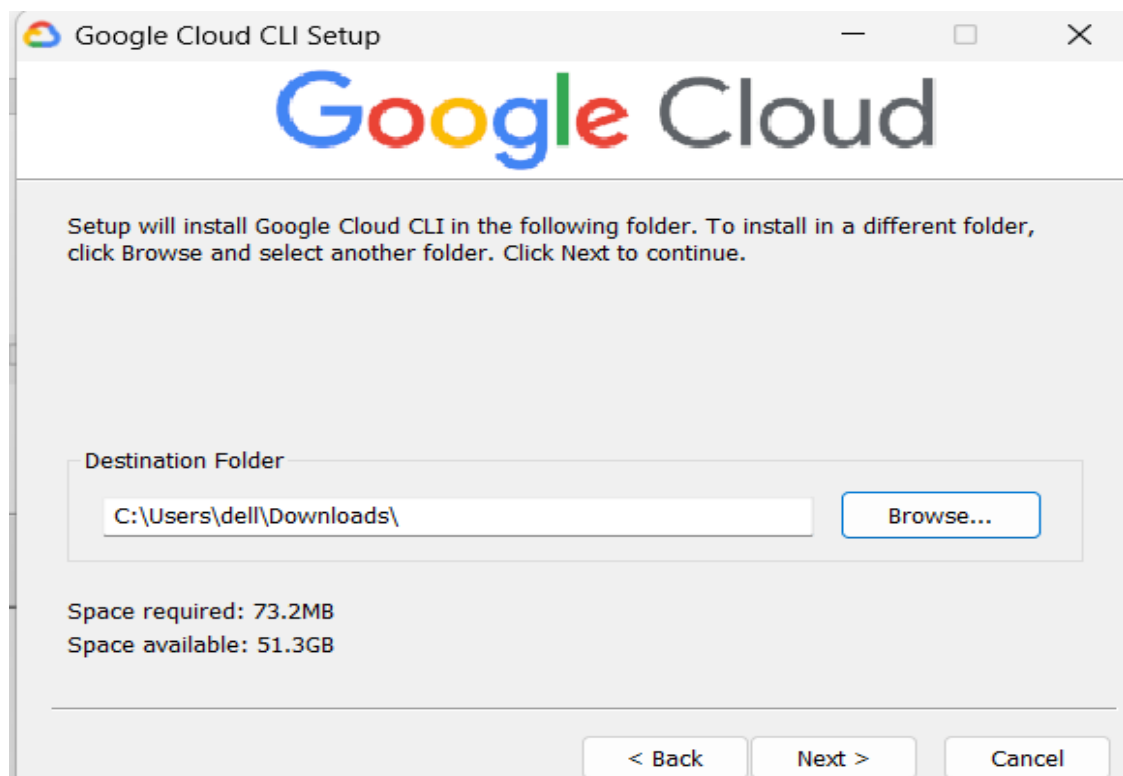


### Step 3: Agreement

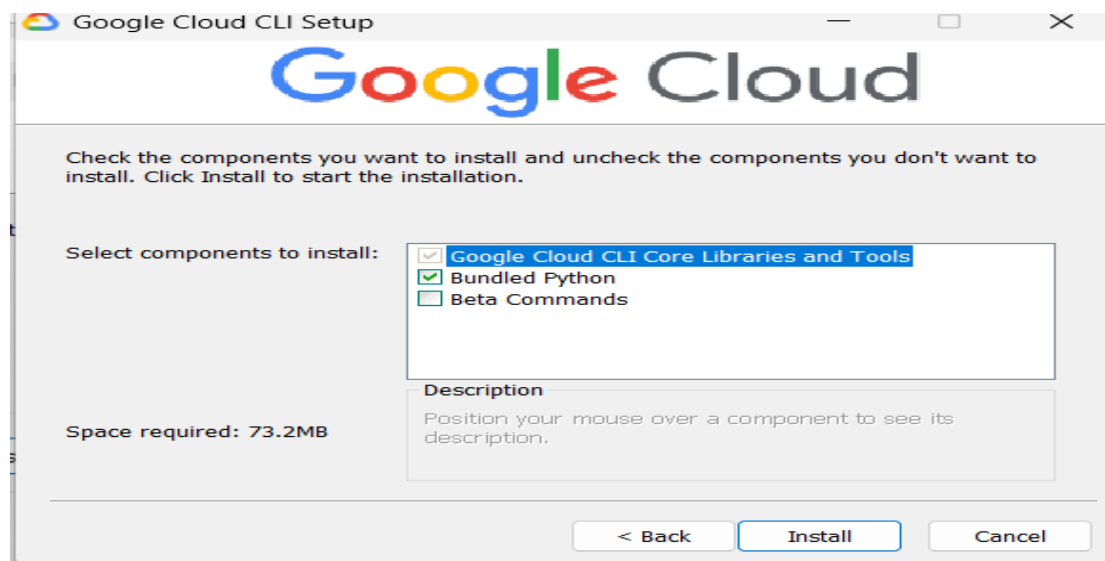
Accept the terms and conditions



### Step 4 : Select Destination Folder

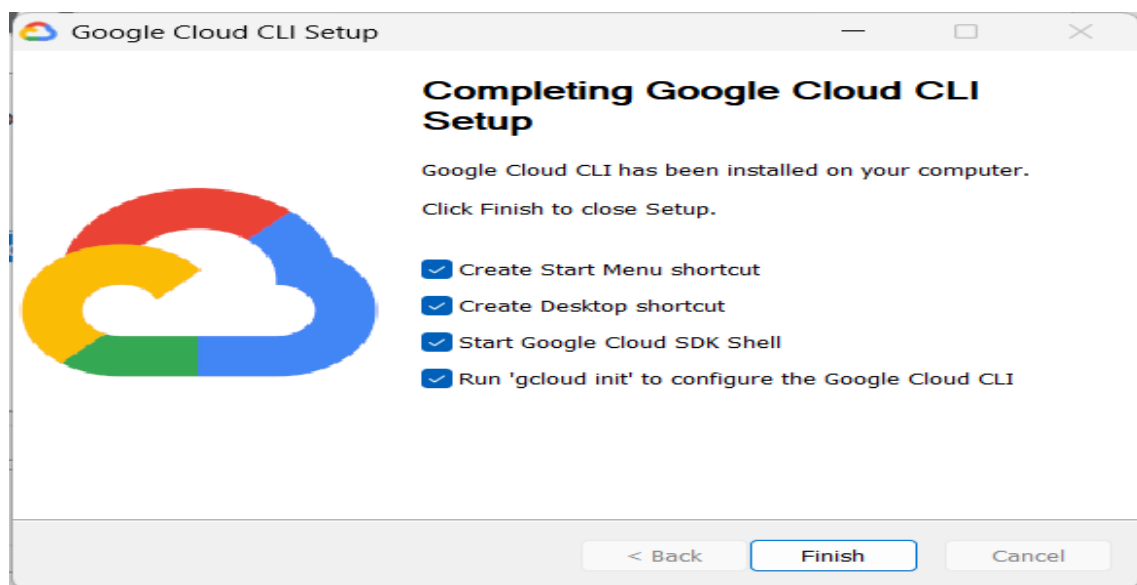


### Step 5: Select components to install



### Step 6: Installation Successful

Once the installation is done click on finish



### Step 7: Initialize gcloud and Login to Google Account

After installation the black command window opens automatically and runs gcloud init. The following message appears:

Welcome to the Google Cloud CLI!

Your current configuration has been set to: [default]

Network diagnostic passed (1/1 checks passed).

You must sign in to continue.

Type Y and press Enter. The browser opens automatically. Login with Gmail account and allow all permissions. The browser displays the message "You are now

authenticated with the gcloud CLI!" confirming successful login. Return to the black command window.

```
Welcome to the Google Cloud CLI! Run "gcloud -h" to get the list of available commands.
---
Welcome! This command will take you through the configuration of gcloud.

Your current configuration has been set to: [default]

You can skip diagnostics next time by using the following flag:
  gcloud init --skip-diagnostics

Network diagnostic detects and fixes local network connection issues.

work connection...done.
Reachability Check passed.
Network diagnostic passed (1/1 checks passed).

You must sign in to continue. Would you like to sign in (selecting "Y" will open your browser for complete authentication) (Y/n)? y

Your browser has been opened to visit:

  https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.ap
%2F&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fw
pis.com%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.log
www.googleapis.com%2Fauth%2Faccounts.reauth&state=DHs6ij6oh8YqgGemR697KZgzdFKLwy&access_t
PGw&code_challenge_method=S256
```

## Step 8 : Select Google Cloud Project

After login the command window shows a list of existing projects. Select the fire-noc-app project by typing its number and pressing Enter.

```
Pick cloud project to use:
[1] fire-department-efe80
[2] fire-noc-app
[3] reflecting-node-456408-b3
[4] studysprint-701c2
[5] studysprint-be12f
[6] Enter a project ID
[7] Create a new project
Please enter numeric choice or text value (must exactly match list item): 2

Your current project has been set to: [fire-noc-app].

Not setting default zone/region (this feature makes it easier to use
[gcloud compute] by setting an appropriate default value for the
--zone and --region flag).
See https://cloud.google.com/compute/docs/gcloud-compute section on how to set
default compute region and zone manually. If you would like [gcloud init] to be
able to do this for you the next time you run it, make sure the
Compute Engine API is enabled for your project on the
https://console.developers.google.com/apis page.

Created a default .boto configuration file at [C:\Users\dell\.boto]. See this file and
[https://cloud.google.com/storage/docs/gsutil/commands/config] for more
information about configuring Google Cloud Storage.
The Google Cloud CLI is configured and ready to use!

* Commands that require authentication will use prakrati1611@gmail.com by default
* Commands will reference project 'fire-noc-app' by default
Run 'gcloud help config' to learn how to change individual settings

This gcloud configuration is called [default]. You can create additional configurations if you work with multiple accounts.
Run 'gcloud topic configurations' to learn more.

Some things to try next:

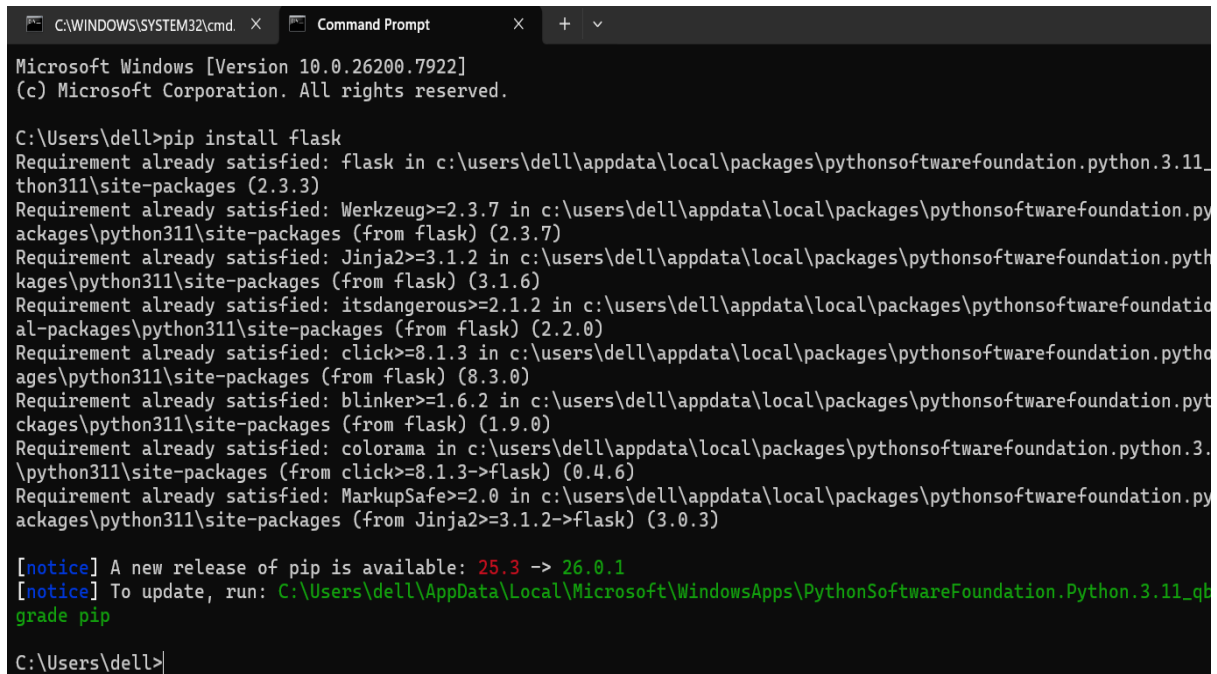
* Run 'gcloud --help' to see the Cloud Platform services you can interact with. And run 'gcloud help COMMAND' to get help for a command.
* Run 'gcloud topic --help' to learn about advanced features of the CLI like arg files and output formatting
* Run 'gcloud cheat-sheet' to see a roster of go-to 'gcloud' commands.

C:\Users\dell\AppData\Local\Google\Cloud SDK>
```

## Step 9: Install Flask

Open Command Prompt and install Flask using pip if not installed

```
pip install flask
```



```
C:\WINDOWS\SYSTEM32\cmd. X Command Prompt X + v
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dell>pip install flask
Requirement already satisfied: flask in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.python.3.11_
thon311\site-packages (2.3.3)
Requirement already satisfied: Werkzeug>=2.3.7 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.py
ackages\python311\site-packages (from flask) (2.3.7)
Requirement already satisfied: Jinja2>=3.1.2 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.pyth
kages\python311\site-packages (from flask) (3.1.6)
Requirement already satisfied: itsdangerous>=2.1.2 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundatio
al-packages\python311\site-packages (from flask) (2.2.0)
Requirement already satisfied: click>=8.1.3 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.pytho
ages\python311\site-packages (from flask) (8.3.0)
Requirement already satisfied: blinker>=1.6.2 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.pyt
ckages\python311\site-packages (from flask) (1.9.0)
Requirement already satisfied: colorama in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.python.3.
\python311\site-packages (from click>=8.1.3->flask) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.py
ackages\python311\site-packages (from Jinja2>=3.1.2->flask) (3.0.3)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: C:\Users\dell\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qb
grade pip

C:\Users\dell>
```

## Step 10: Open Project Folder

Open Command Prompt and navigate to the Fire NOC project folder using the cd command:

```
cd C:\Users\dell\Desktop\firenoc-demo
```

## Step 11 : Create app.yaml File

Inside the project folder, create a new text file and name it app.yaml. Open it with Notepad and type the following:

```
runtime: python310

instance_class: F1

handlers:
- url: /static
  static_dir: static

- url: /.
  script: auto
...
```

```
! app.yaml
1  runtime: python310
2
3  instance_class: F1
4
5  handlers:
6  - url: /static
7    | static_dir: static
8
9  - url: /.
10 | script: auto
11
12
13
```

## Step 12 : Create requirements.txt File

Create a file named requirements.txt inside the firenoc-demo folder with the following content. This file lists all Python libraries needed to run the application:

```
requirements.txt
1  Flask==3.0.0
2
```

## Step 13: Verify Project Folder Structure

| Name         | Date modified       | Type               | Size |
|--------------|---------------------|--------------------|------|
| templates    | 19-03-2026 07:43 PM | File folder        |      |
| app          | 19-03-2026 07:37 PM | Python Source File | 1 KB |
| app.yaml     | 19-03-2026 07:44 PM | YAML File          | 0 KB |
| requirements | 19-03-2026 07:47 PM | Text Document      | 1 KB |

## Step 14 : Run the application locally

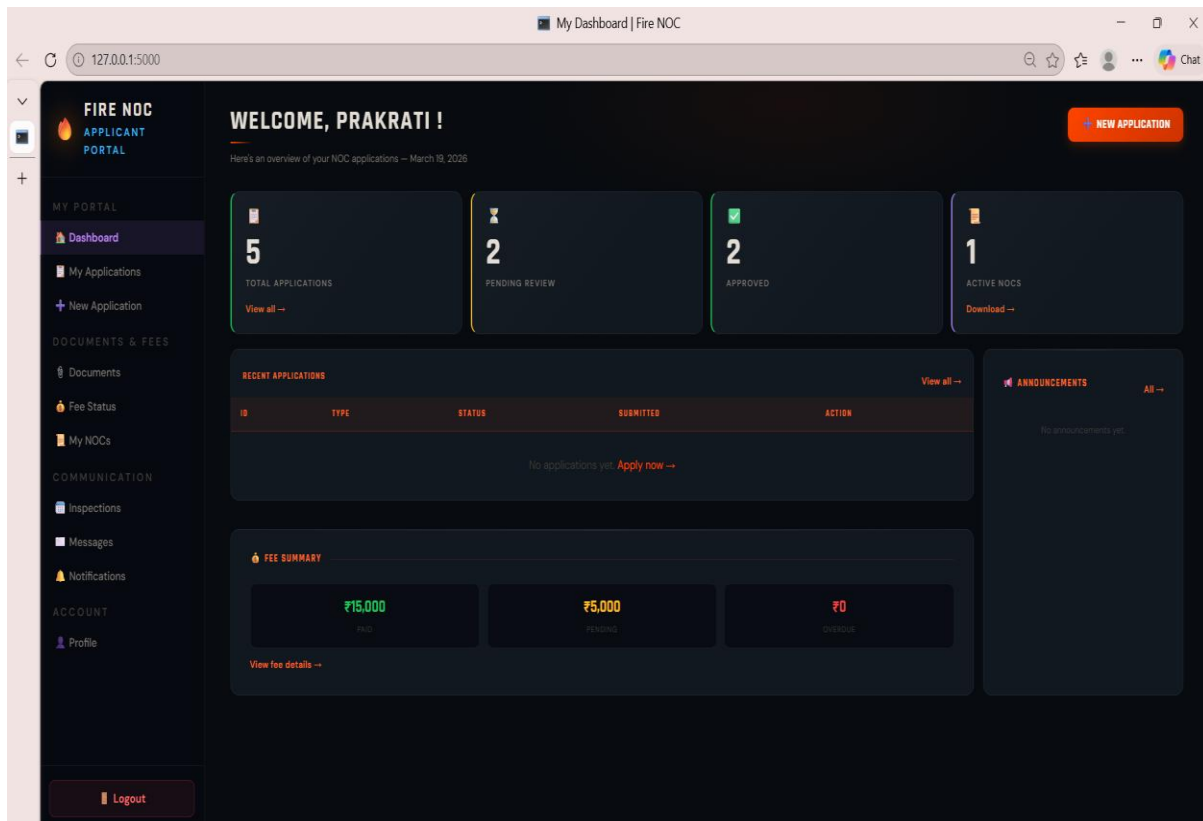
Run the Flask application:

```
python app.py
```

```
C:\Users\dell\Desktop\firenoc-demo>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 100-138-245
```

## Step 15: Open Application in Browser

Open Chrome and visit: `` http://127.0.0.1:5000



## RESULT

Google App Engine development environment was successfully installed on Windows using Google Cloud SDK. The Real-Time Fire NOC Monitoring web application was successfully created using Python and Flask. The applicant dashboard was rendered correctly at <http://127.0.0.1:5000> displaying dynamic KPI statistics, fee summaries, navigation links, and announcements as expected. The application demonstrated a complete working web application built on the Google App Engine framework.

## EXPERIMENT – 4

**OBJECTIVE:** Find a procedure to transfer the file from one VM to another VM.

### THEORY

**File Transfer between Virtual Machines** is the process of sharing or moving files from one VM to another running on the same physical host machine. Since each VM operates as an independent system with its own network interface, file transfer requires a network-based or shared storage approach.

**Python HTTP Server** is a built-in module in Python that allows any machine to act as a simple web server. It serves files from a directory over HTTP protocol, making them accessible to other machines on the same network via a web browser or download command.

**IP Address** is a unique numerical label assigned to each device on a network. In a Host-Only network, VirtualBox assigns each VM a unique IP address (e.g., 192.168.56.101, 192.168.56.102) allowing them to identify and communicate with each other.

### Key Concepts:

**HTTP (HyperText Transfer Protocol)** — The protocol used for transferring data over the web. Python's HTTP server uses this protocol to serve files.

**wget command** — A Linux command used to download files from the internet or a local network server.

**Port** — A communication endpoint. Python HTTP server by default uses port 8000.

### NAT vs Host-Only Network:

- **NAT** — VMs can access internet but cannot communicate with each other
- **Host-Only** — VMs can communicate with each other but cannot access internet

### PROCEDURE

#### Step 1: Download and Set Up Ubuntu 22.04 ISO

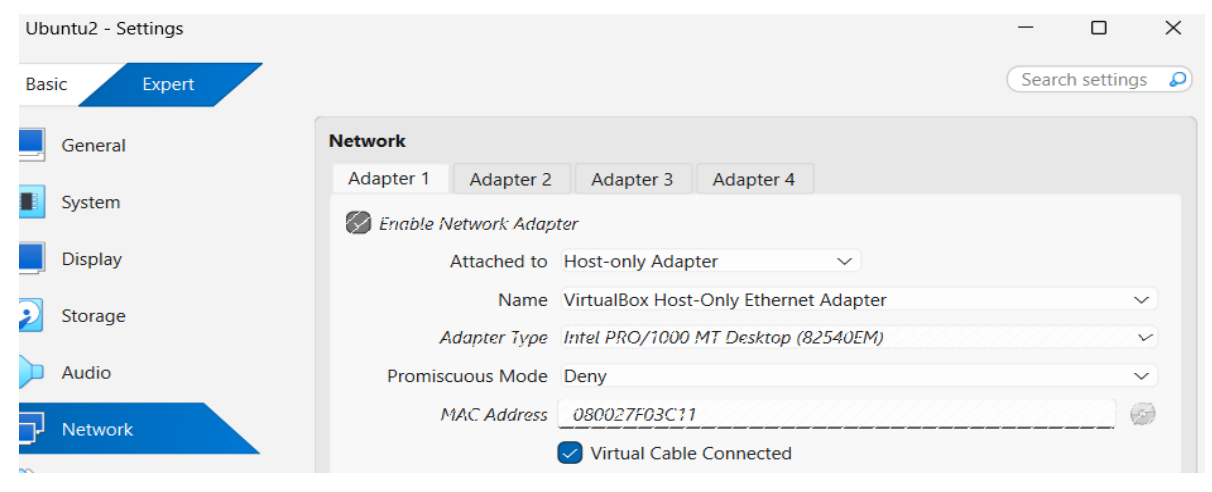
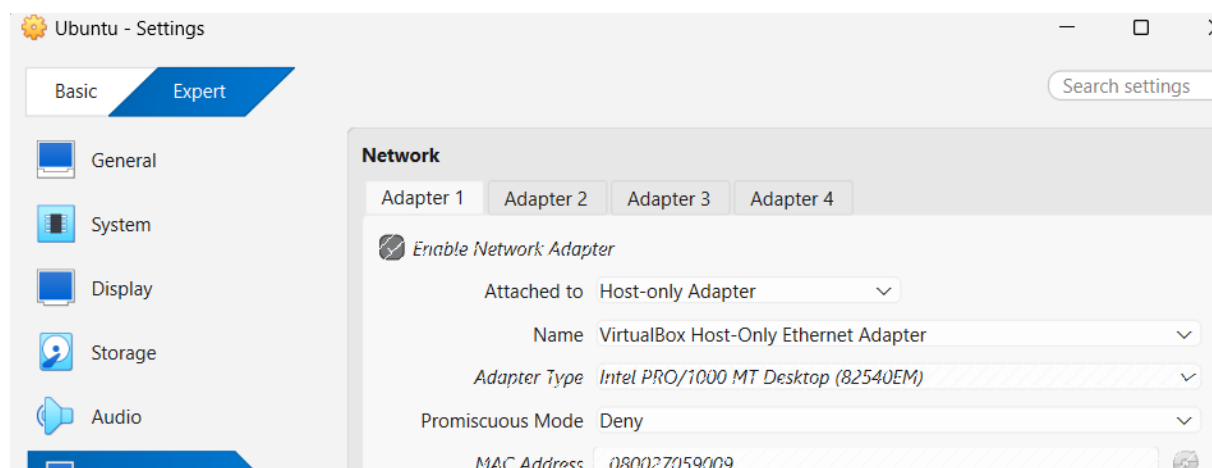
Download Ubuntu 22.04 LTS ISO from the official Ubuntu website. Create two Virtual Machines in VirtualBox — VM1 (Ubuntu) and VM2 (Ubuntu2) — using the same ISO file.



## Step 2: Configure Host-Only Network for Both VMs

Power off both VMs. For each VM:

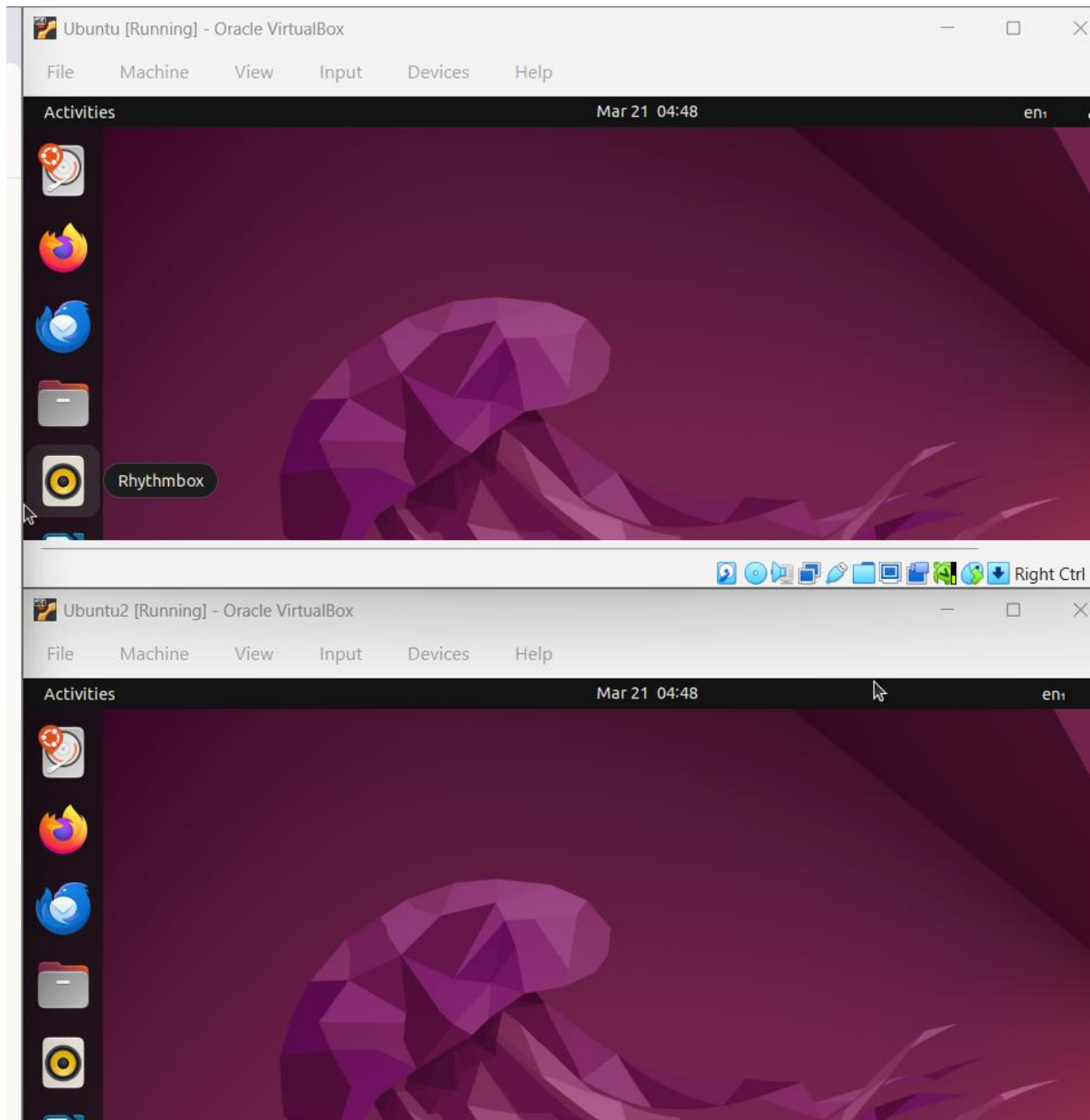
- Click **Settings** → **Network**
- Change **Adapter 1** from **NAT** to **Host-only Adapter**
- Click **OK**





### Step 3: Start Both VMs

Start both Ubuntu VM1 and Ubuntu2 VM2. Wait for both desktops to load completely.

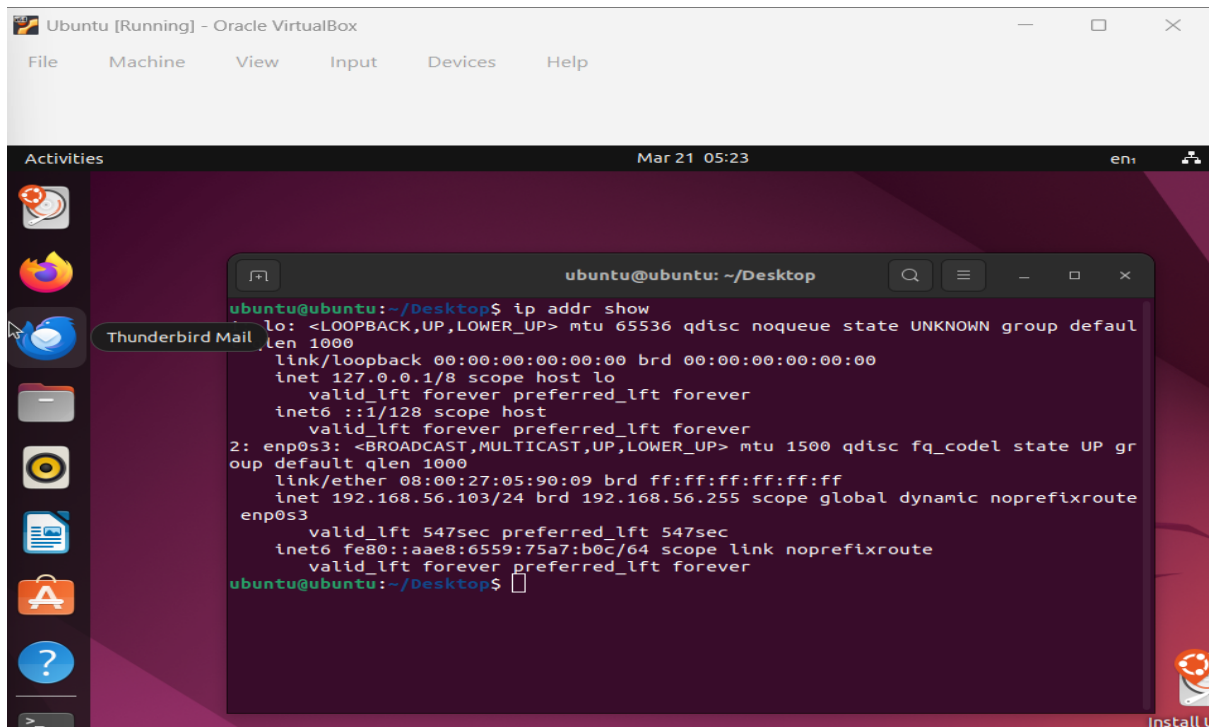


### Step 4: Get IP Address of VM1

Open Terminal in **Ubuntu VM1** and type:

```
ip addr show
```

Note the IP address of VM1 (e.g., 192.168.56.101)



The screenshot shows a terminal window titled 'ubuntu@ubuntu: ~/Desktop' within an Oracle VM VirtualBox environment. The terminal displays the output of the 'ip addr show' command. It shows the loopback interface 'lo' with IP 127.0.0.1 and the ethernet interface 'enp0s3' with IP 192.168.56.103. The desktop background is Ubuntu's default purple theme, and the left sidebar shows icons for Thunderbird Mail, Files, and other applications.

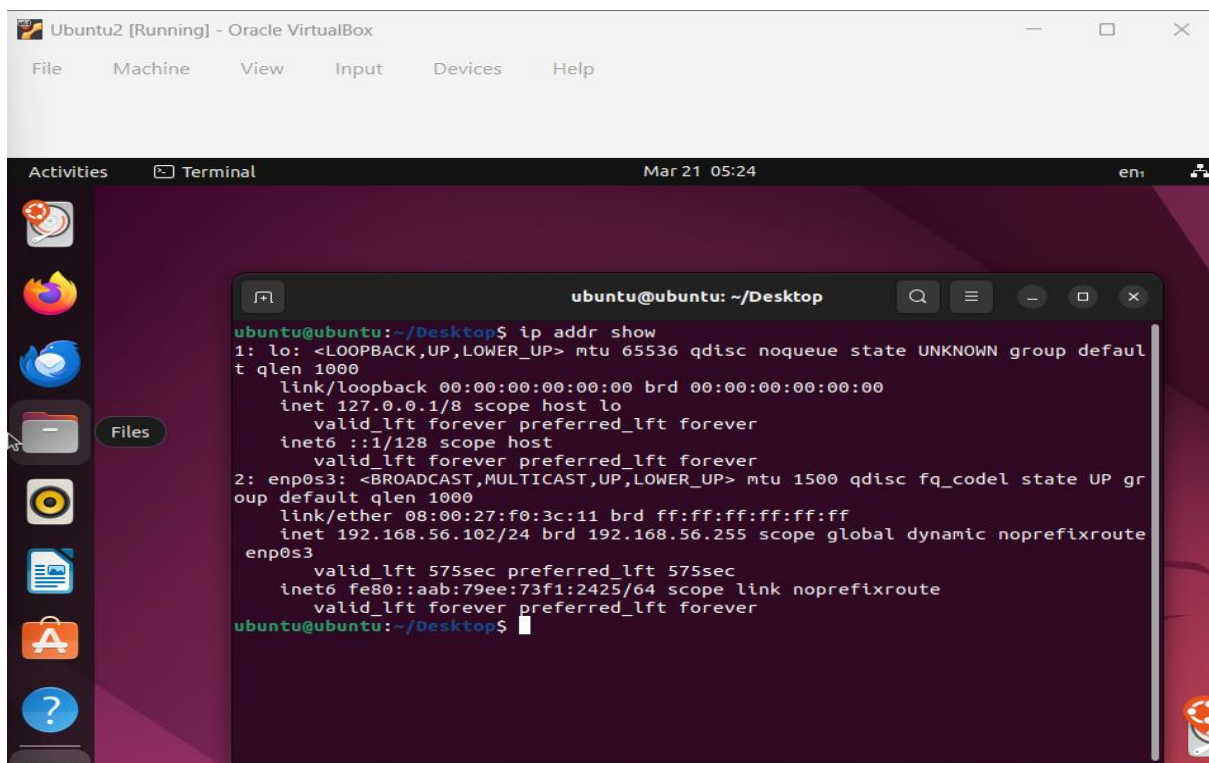
```
ubuntu@ubuntu:~/Desktop$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:05:90:09 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 547sec preferred_lft 547sec
    inet6 fe80::a8e8:6559:75a7:b0c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
ubuntu@ubuntu:~/Desktop$
```

## Step 5: Get IP Address of VM2

Open Terminal in **Ubuntu VM2** and type:

```
ip addr show
```

Note the IP address of VM2 (e.g., 192.168.56.102)



The screenshot shows a terminal window titled 'ubuntu@ubuntu: ~/Desktop' within an Oracle VM VirtualBox environment. The terminal displays the output of the 'ip addr show' command. It shows the loopback interface 'lo' with IP 127.0.0.1 and the ethernet interface 'enp0s3' with IP 192.168.56.102. The desktop background is Ubuntu's default purple theme, and the left sidebar shows icons for Files and other applications.

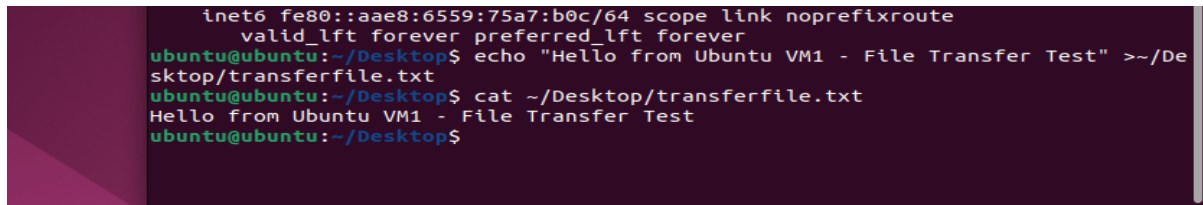
```
ubuntu@ubuntu:~/Desktop$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f0:3c:11 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 575sec preferred_lft 575sec
    inet6 fe80::aab:79ee:73f1:2425/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
ubuntu@ubuntu:~/Desktop$
```

## Step 6: Create a File in VM1

In Ubuntu VM1 terminal, create a text file to transfer:

```
echo "Hello from Ubuntu VM1 - File Transfer Test" >
~/Desktop/transferfile.txt
```

Verify the file was created: `cat ~/Desktop/transferfile.txt`

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop\$'. The user enters 'echo "Hello from Ubuntu VM1 - File Transfer Test" > ~/Desktop/transferfile.txt'. The prompt changes to 'ubuntu@ubuntu:~/Desktop\$' and the user enters 'cat ~/Desktop/transferfile.txt'. The output is 'Hello from Ubuntu VM1 - File Transfer Test'.

```
inet6 fe80::aae8:6559:75a7:b0c/64 scope link noprefixroute
valid_lft forever preferred_lft forever
ubuntu@ubuntu:~/Desktop$ echo "Hello from Ubuntu VM1 - File Transfer Test" > ~/Desktop/transferfile.txt
ubuntu@ubuntu:~/Desktop$ cat ~/Desktop/transferfile.txt
Hello from Ubuntu VM1 - File Transfer Test
ubuntu@ubuntu:~/Desktop$
```

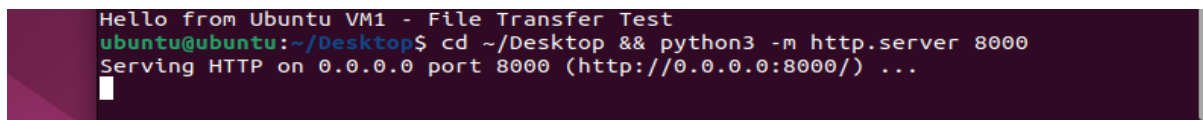
## Step 7: Start Python HTTP Server in VM1

Navigate to the Desktop directory and start Python HTTP server:

```
cd ~/Desktop -m http.server 8000
```

The terminal shows:

Serving HTTP on 0.0.0.0 port 8000

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop\$'. The user enters 'cd ~/Desktop && python3 -m http.server 8000'. The output is 'Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...'.

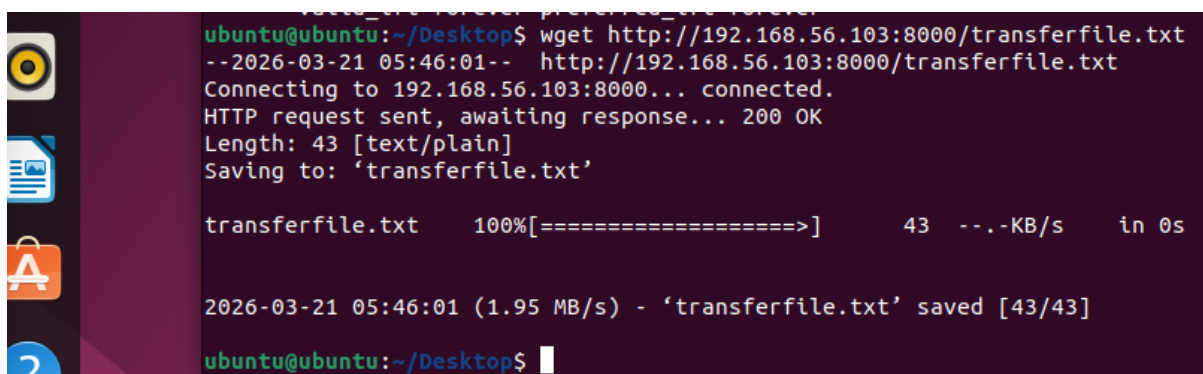
```
Hello from Ubuntu VM1 - File Transfer Test
ubuntu@ubuntu:~/Desktop$ cd ~/Desktop && python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

## Step 8: Download File from VM2

In Ubuntu VM2 terminal, type the following command using VM1's IP address:

```
wget http://192.168.56.101:8000/transferfile.txt
```

The file downloads from VM1 to VM2.

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop\$'. The user enters 'wget http://192.168.56.103:8000/transferfile.txt'. The output shows the download progress: '--2026-03-21 05:46:01-- http://192.168.56.103:8000/transferfile.txt', 'Connecting to 192.168.56.103:8000... connected.', 'HTTP request sent, awaiting response... 200 OK', 'Length: 43 [text/plain]', 'Saving to: 'transferfile.txt'', a progress bar, and '2026-03-21 05:46:01 (1.95 MB/s) - 'transferfile.txt' saved [43/43]'. The prompt changes to 'ubuntu@ubuntu:~/Desktop\$'.

```
ubuntu@ubuntu:~/Desktop$ wget http://192.168.56.103:8000/transferfile.txt
--2026-03-21 05:46:01-- http://192.168.56.103:8000/transferfile.txt
Connecting to 192.168.56.103:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 43 [text/plain]
Saving to: 'transferfile.txt'

transferfile.txt  100%[=====>]          43  --.-KB/s   in 0s

2026-03-21 05:46:01 (1.95 MB/s) - 'transferfile.txt' saved [43/43]

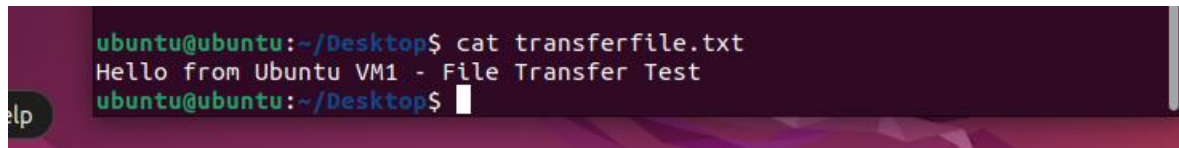
ubuntu@ubuntu:~/Desktop$
```

### Step 9: Verify File Transfer

In Ubuntu VM2 terminal, verify the file was received:

```
cat transferfile.txt
```

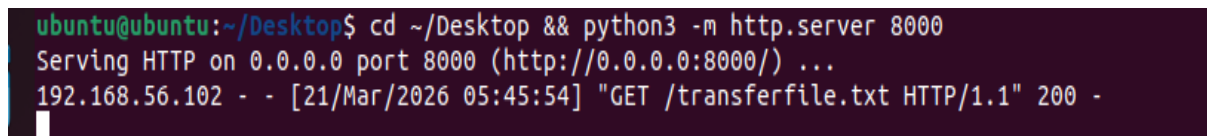
Output: Hello from Ubuntu VM1 - File Transfer Test

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop\$'. The user enters 'cat transferfile.txt'. The output is 'Hello from Ubuntu VM1 - File Transfer Test'. The prompt returns to 'ubuntu@ubuntu:~/Desktop\$' with a cursor.

```
ubuntu@ubuntu:~/Desktop$ cat transferfile.txt
Hello from Ubuntu VM1 - File Transfer Test
ubuntu@ubuntu:~/Desktop$
```

### Step 10: Confirm Transfer in VM1

In VM1 terminal, the Python server log shows the download request from VM2's IP address: 192.168.56.102 - - [date] "GET /transferfile.txt HTTP/1.1" 200 -

A terminal window with a dark purple background. The prompt is 'ubuntu@ubuntu:~/Desktop\$'. The user enters 'cd ~/Desktop && python3 -m http.server 8000'. The output shows the server starting on port 8000 and then receiving a request from 192.168.56.102.

```
ubuntu@ubuntu:~/Desktop$ cd ~/Desktop && python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.56.102 - - [21/Mar/2026 05:45:54] "GET /transferfile.txt HTTP/1.1" 200 -
```

## RESULT

A file was successfully transferred from Ubuntu Virtual Machine 1 (VM1) to Ubuntu Virtual Machine 2 (VM2) using Python HTTP Server over a Host-Only Network in Oracle VirtualBox. The file "**transferfile.txt**" created in VM1 was downloaded in VM2 using the **wget** command, confirming successful file transfer between two Virtual Machines running on the same physical host machine.

## CONCLUSION

This experiment successfully demonstrated the procedure for transferring files between two Virtual Machines using Python's built-in HTTP server module. By configuring both VMs with a Host-Only Network adapter, a private communication channel was established between them. VM1 acted as the **server** hosting the file, while VM2 acted as the **client** downloading it. This method is simple, requires no additional software installation, and effectively demonstrates network-based file transfer between Virtual Machines.